

Machine Learning clásico

Clase 5, parte 2 de 2 · Generalización, clasificación y PCA

Diseño y Optimización de Sistemas Térmicos · ILN223

Mijail Littin

Universidad Técnica Federico Santa María · 2026-2

Brunton y Kutz, Data-Driven Science and Engineering, cap. 4

Plan de la clase

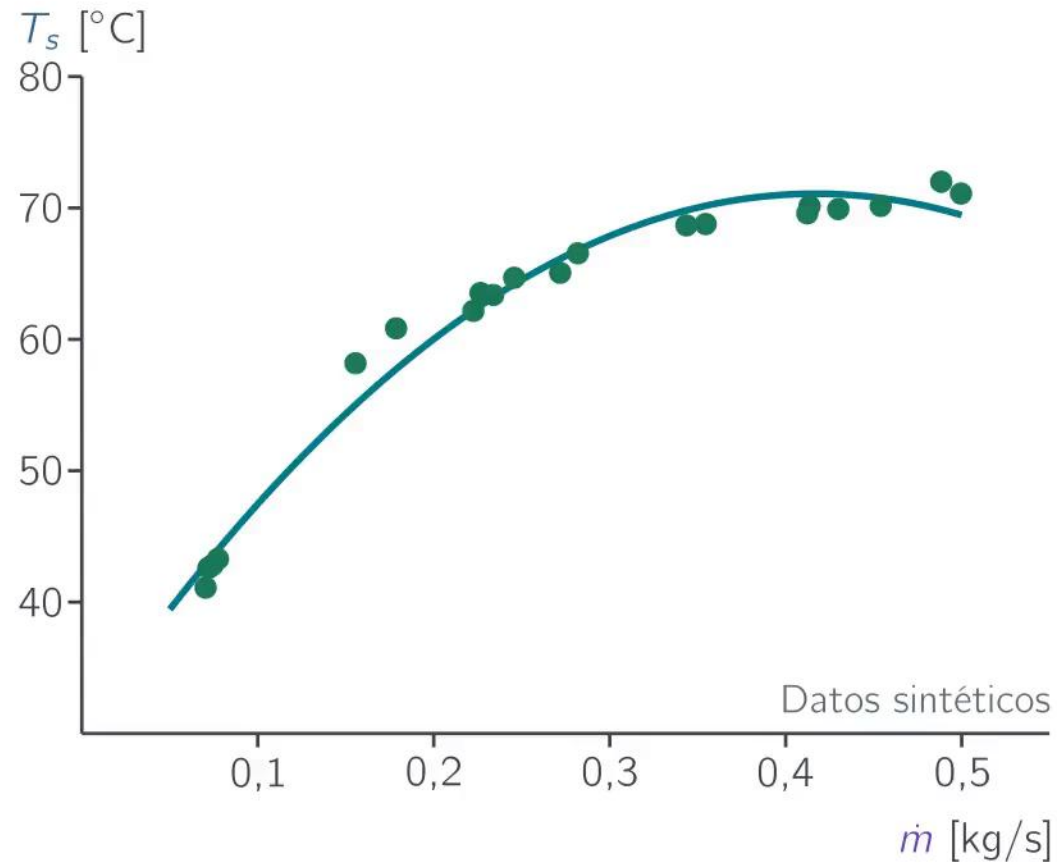
Clase 5

- 1 Regresión y regularización
- 2 Generalización, clasificación y PCA

Hoy

- **Generalización**, sobreajuste y validación
- **Pareto**, error contra complejidad
- **Clasificación**, regresión logística
- **PCA**, reducción de dimensionalidad

Sobreaajuste y subajuste



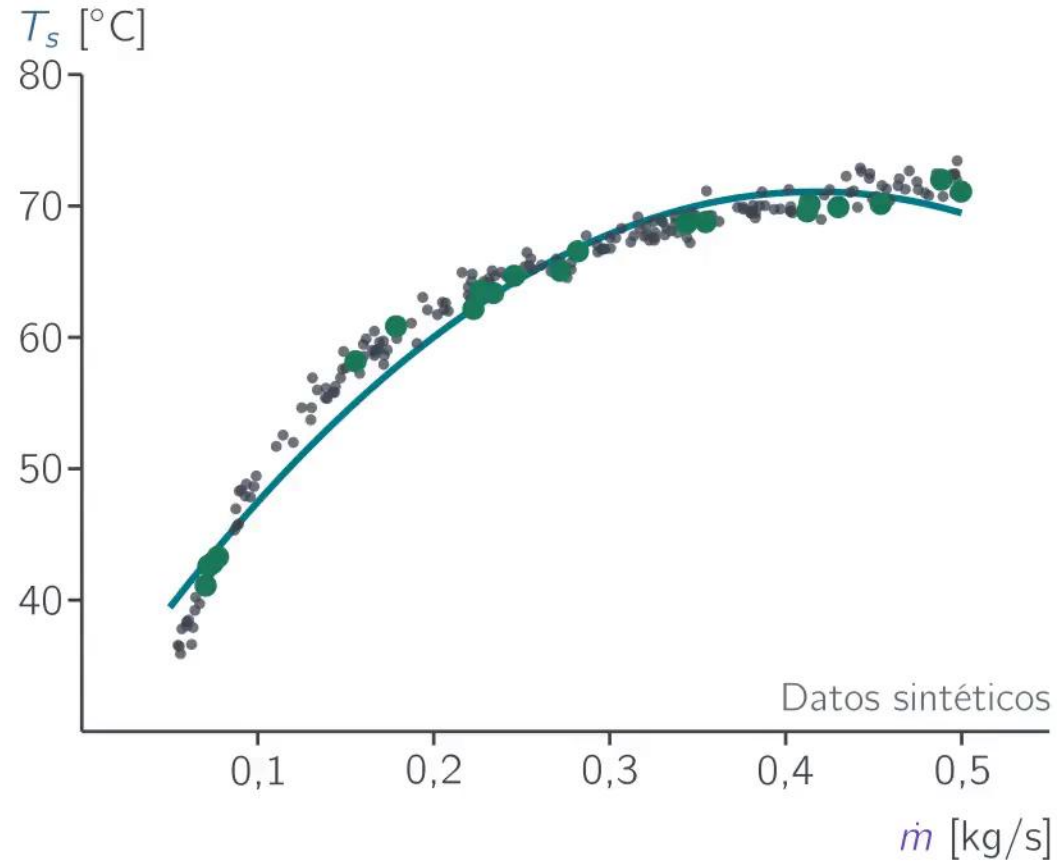
$$\hat{T}_s = x_0 + x_1 z + x_2 z^2$$

$$z = \frac{\dot{m} - 0,275}{0,225}$$

$$ECM = \frac{1}{m} \sum_{i=1}^m (\hat{T}_{s,i} - T_{s,i})^2$$

$$ECM_{\text{entrenamiento}} = \frac{41,3}{20} = 2,06 \text{ (°C)}^2$$

Sobreajuste y subajuste



$$\hat{T}_s = x_0 + x_1 z + x_2 z^2$$

$$z = \frac{\dot{m} - 0,275}{0,225}$$

$$ECM = \frac{1}{m} \sum_{i=1}^m (\hat{T}_{s,i} - T_{s,i})^2$$

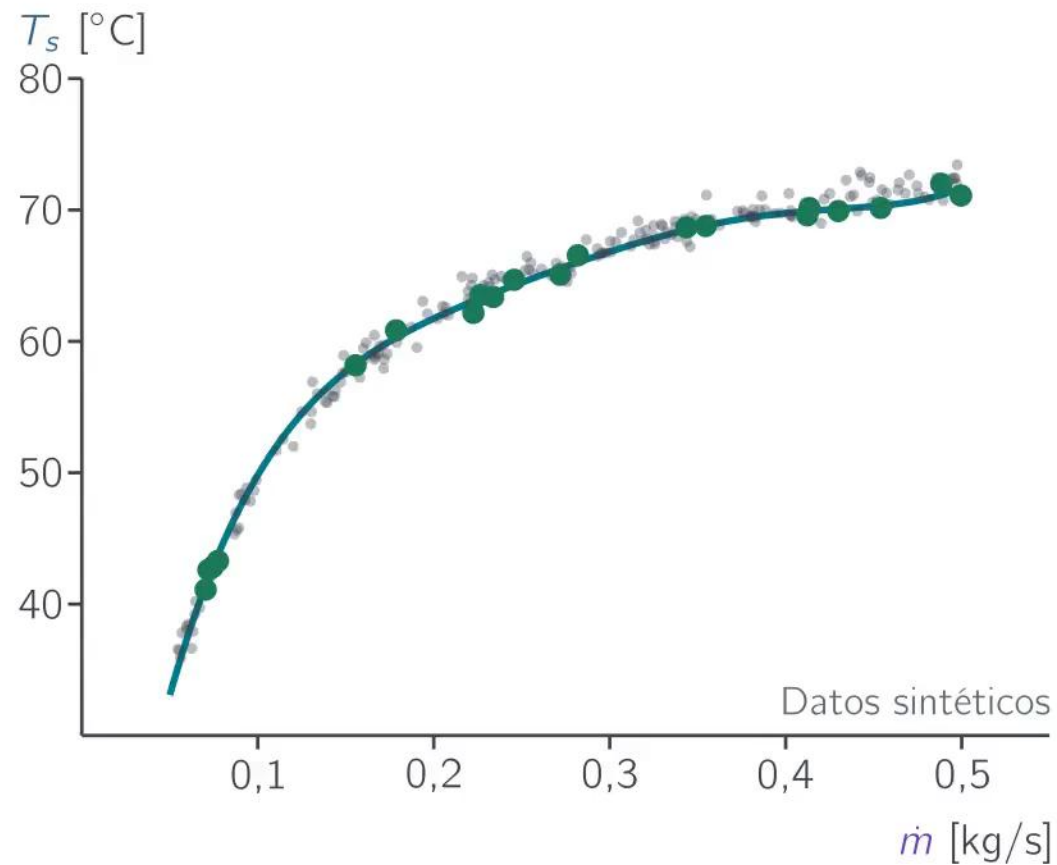
$$ECM_{\text{entrenamiento}} = \frac{41,3}{20} = 2,06 \text{ (°C)}^2$$

$$ECM_{\text{datos nuevos}} = 3,86 \text{ (°C)}^2$$

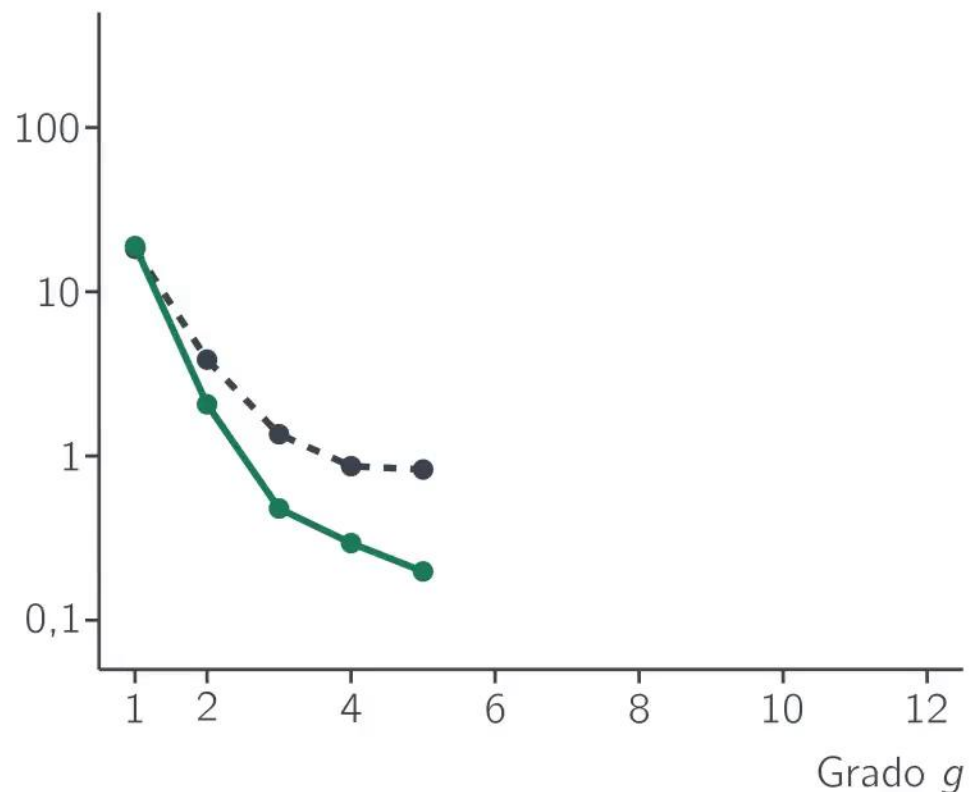
200 datos nuevos, solo como referencia

Sobreajuste y subajuste

$$\hat{T}_s = x_0 + x_1 z + \dots + x_g z^g \quad g = 5$$



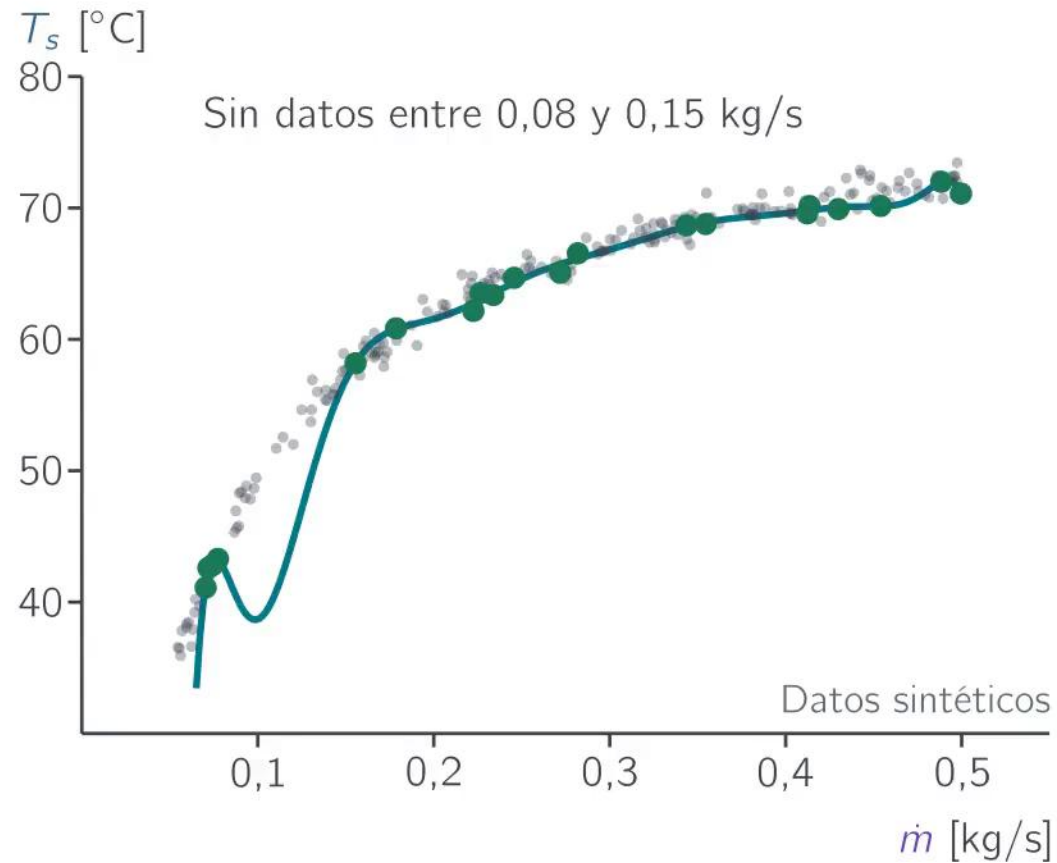
■ Entrenamiento ■ Datos nuevos, referencia
ECM [(°C)²], escala log



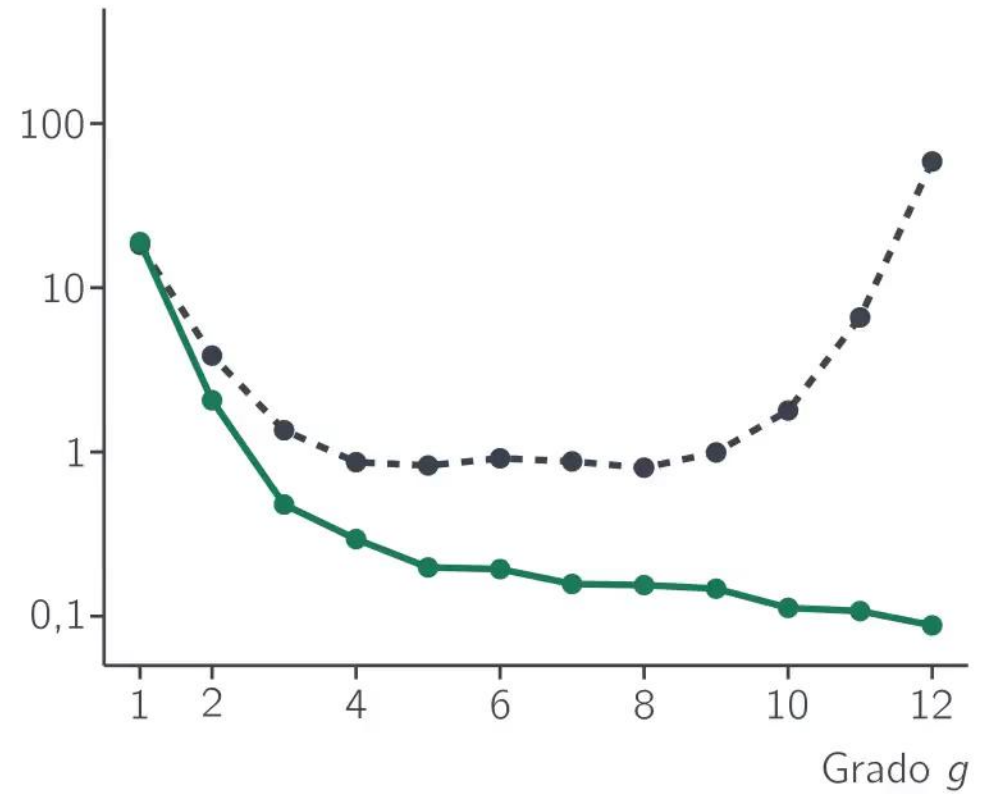
Pregunta de control. Si solo miramos el error de entrenamiento, ¿qué grado elegirían?

Sobreajuste y subajuste

$$\hat{T}_s = x_0 + x_1 z + \dots + x_g z^g \quad g = 12$$

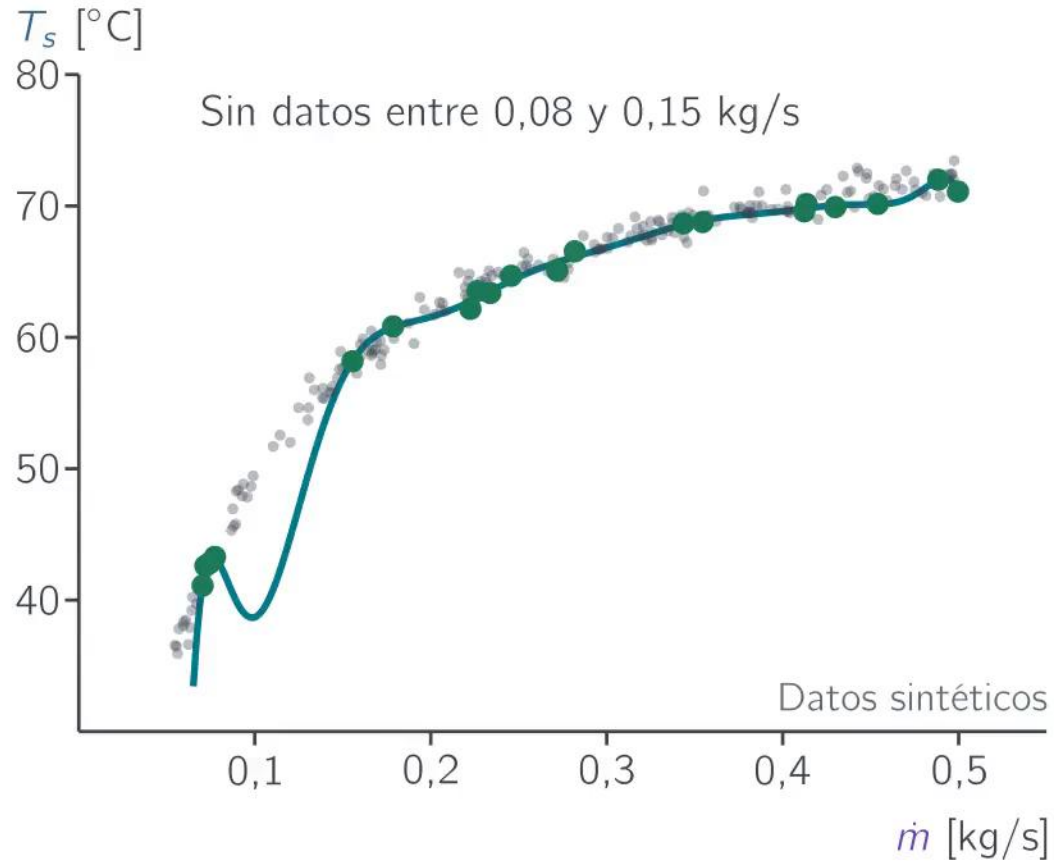


■ Entrenamiento ■ Datos nuevos, referencia
ECM [(°C)²], escala log

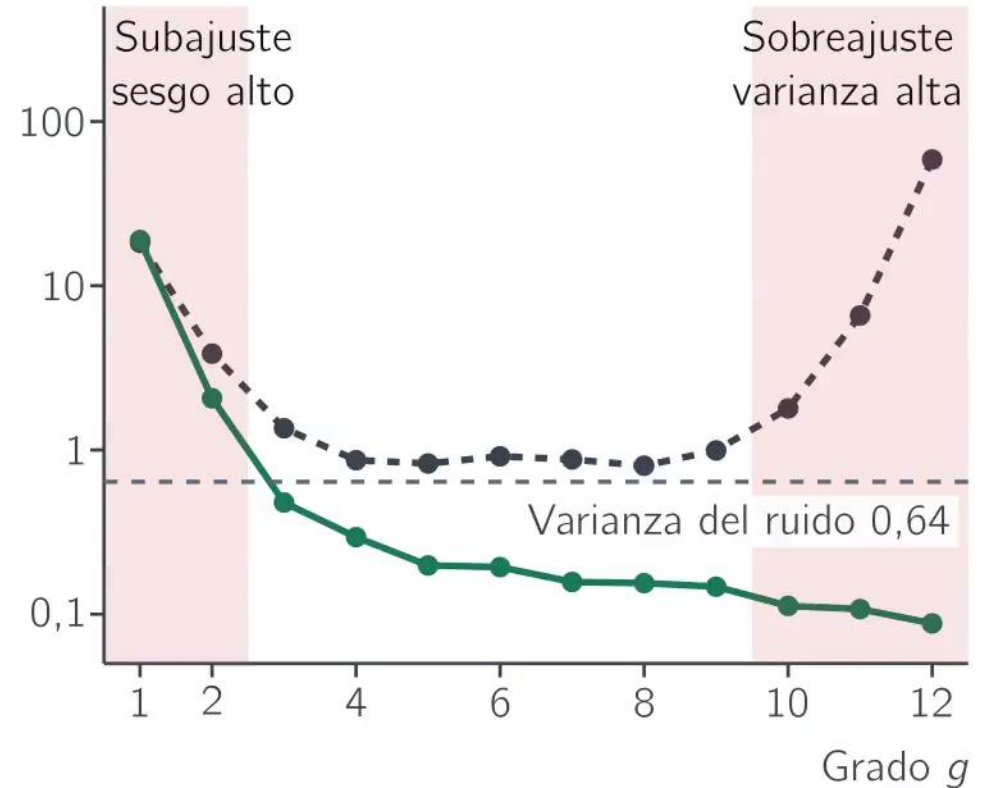


Sobreajuste y subajuste

$$\hat{T}_s = x_0 + x_1 z + \dots + x_g z^g \quad g = 12$$



■ Entrenamiento ■ Datos nuevos, referencia
ECM [(°C)²], escala log

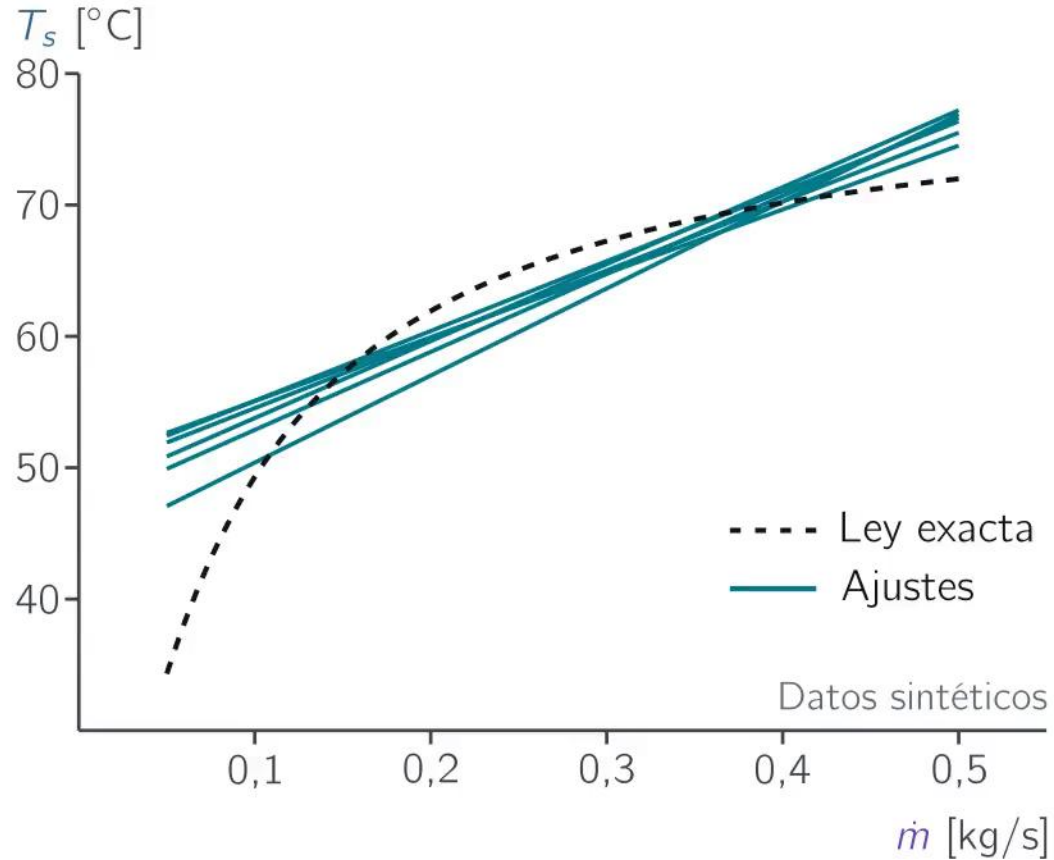


El error de entrenamiento siempre baja. El de datos nuevos tiene un mínimo.

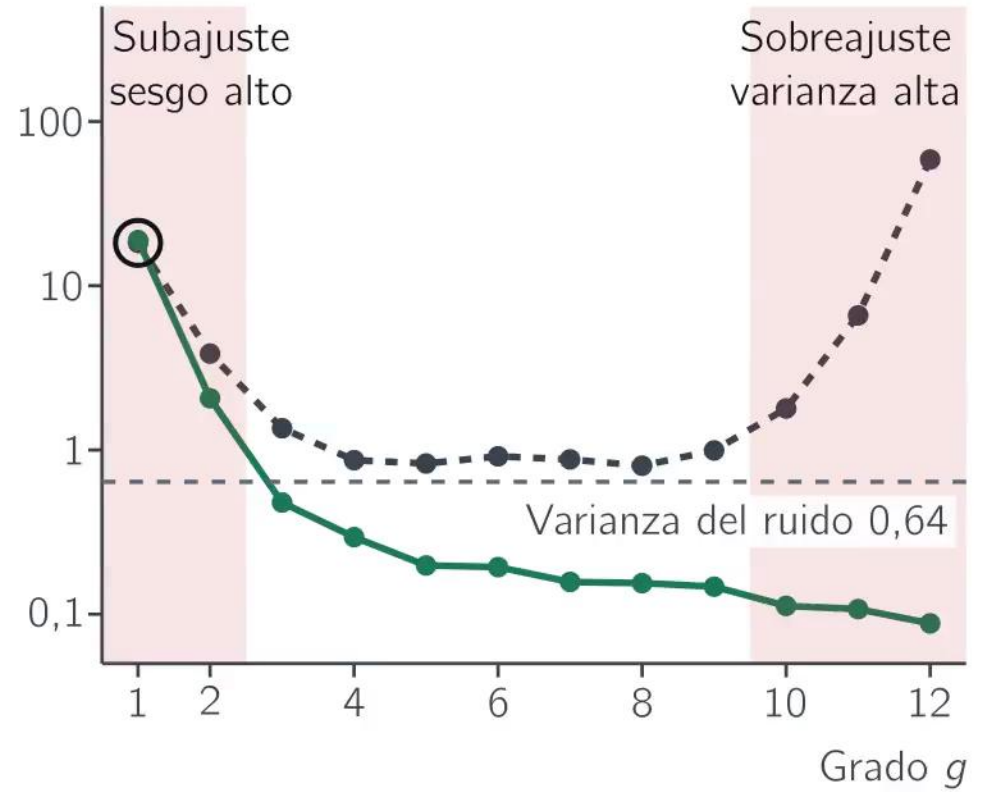
Sobreajuste y subajuste

6 conjuntos de 20 datos

$g = 1$



■ Entrenamiento ■ Datos nuevos, referencia
ECM [(°C)²], escala log

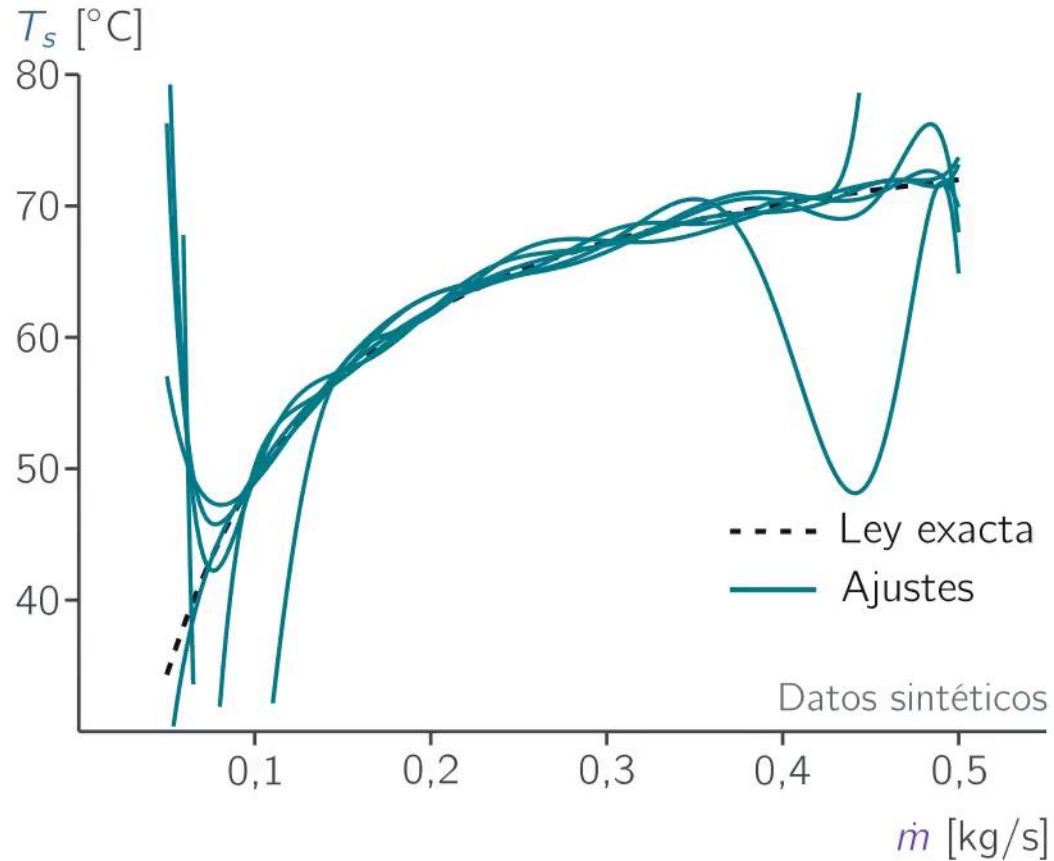


Sesgo, error sistemático del modelo.

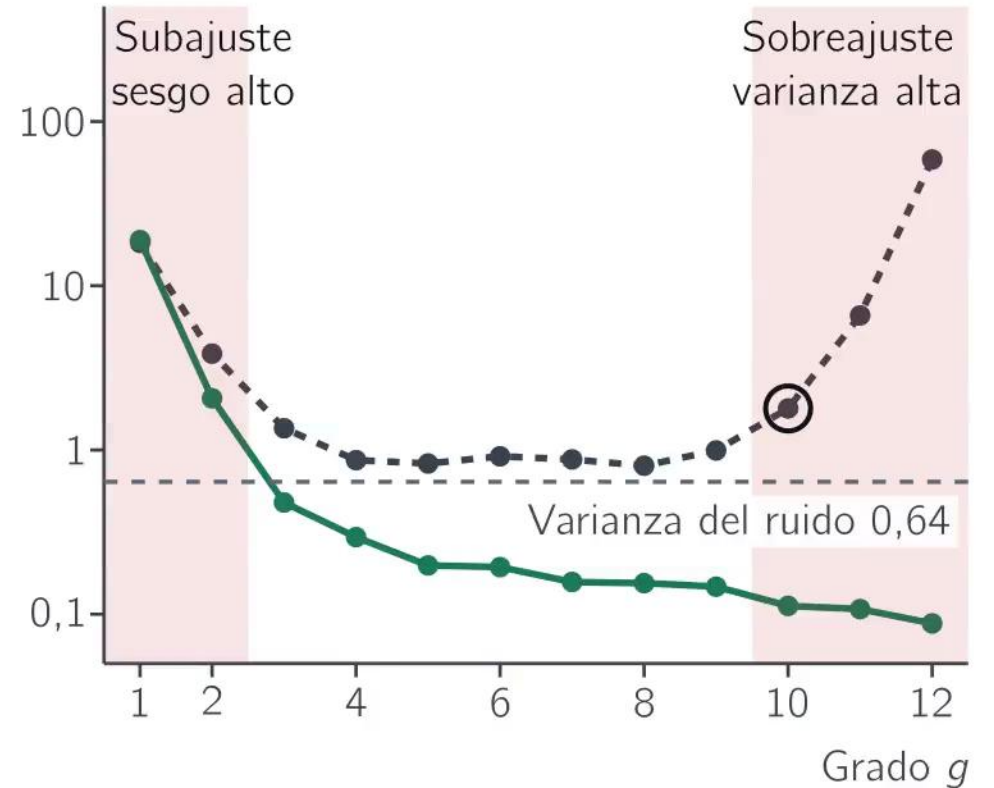
Sobreajuste y subajuste

6 conjuntos de 20 datos

$g = 10$



■ Entrenamiento ■ Datos nuevos, referencia
ECM [(°C)²], escala log

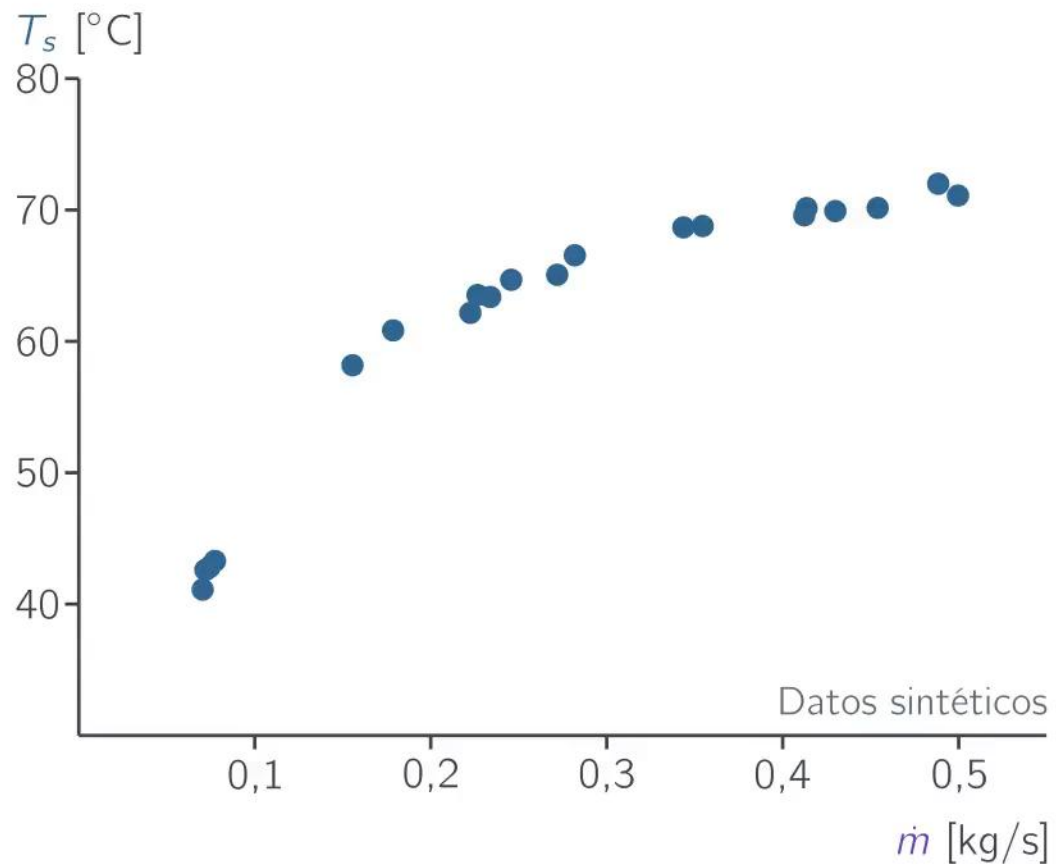


En la práctica no tenemos la curva de datos nuevos.

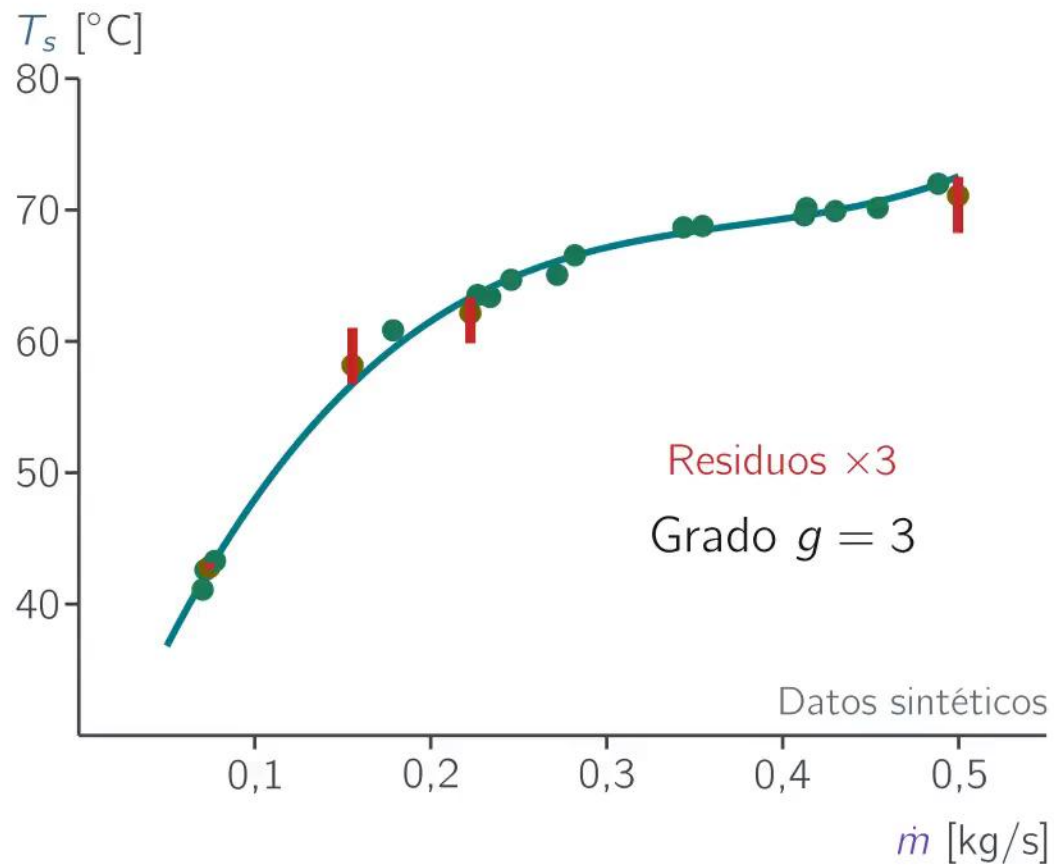
Validación cruzada



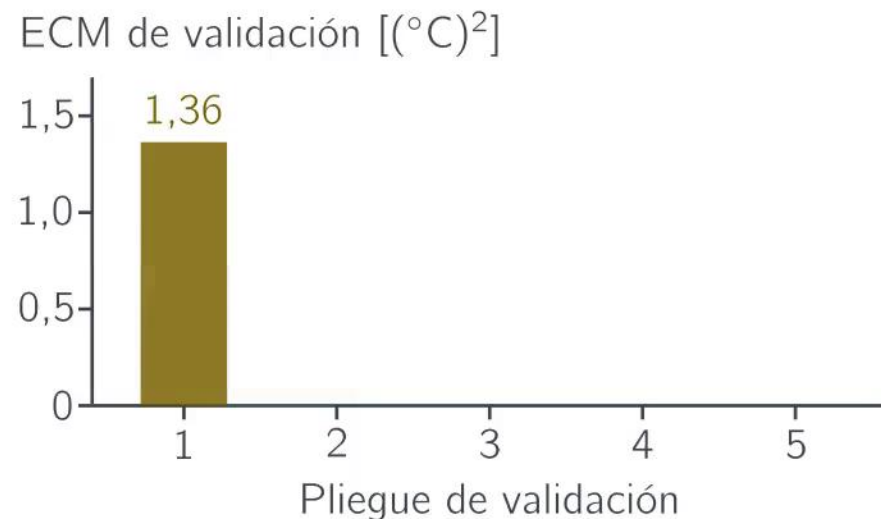
1 $k = 5$ pliegues (folds) al azar



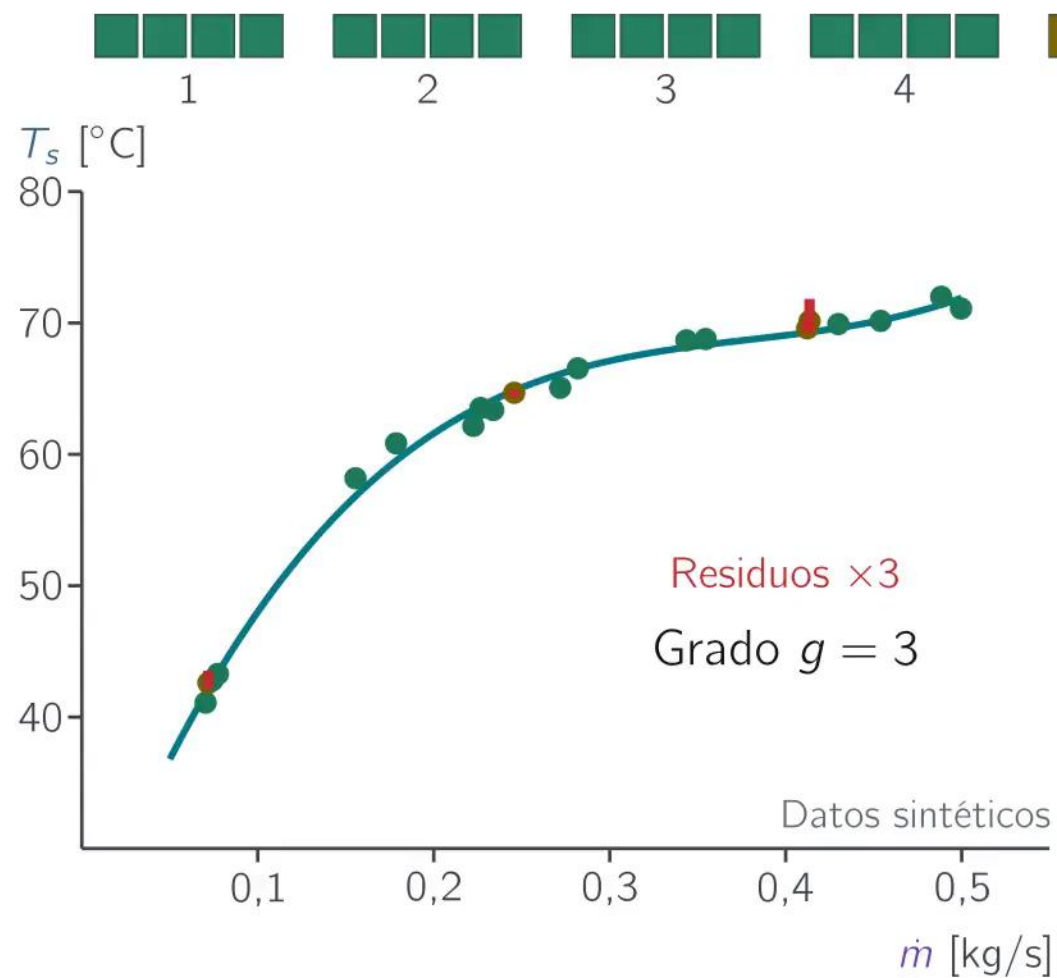
Validación cruzada



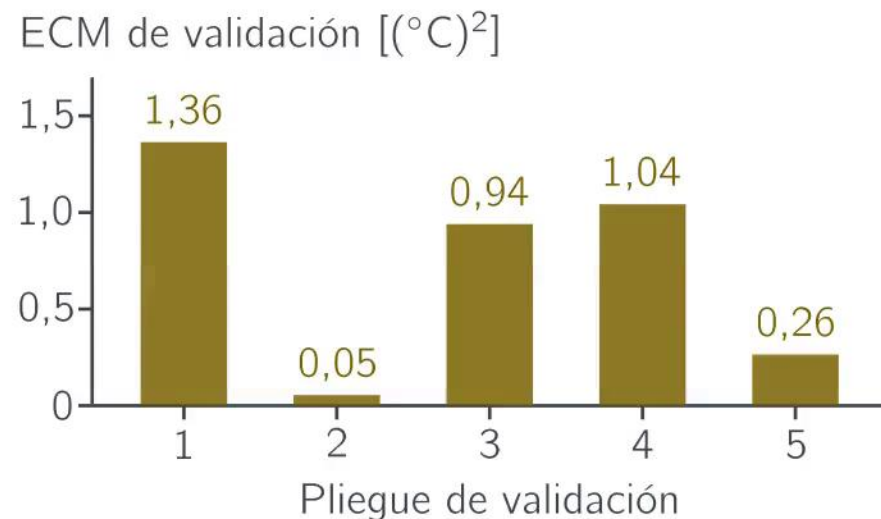
- 1 $k = 5$ pliegues (folds) al azar
- 2 Entrenar con $k - 1$, validar con 1



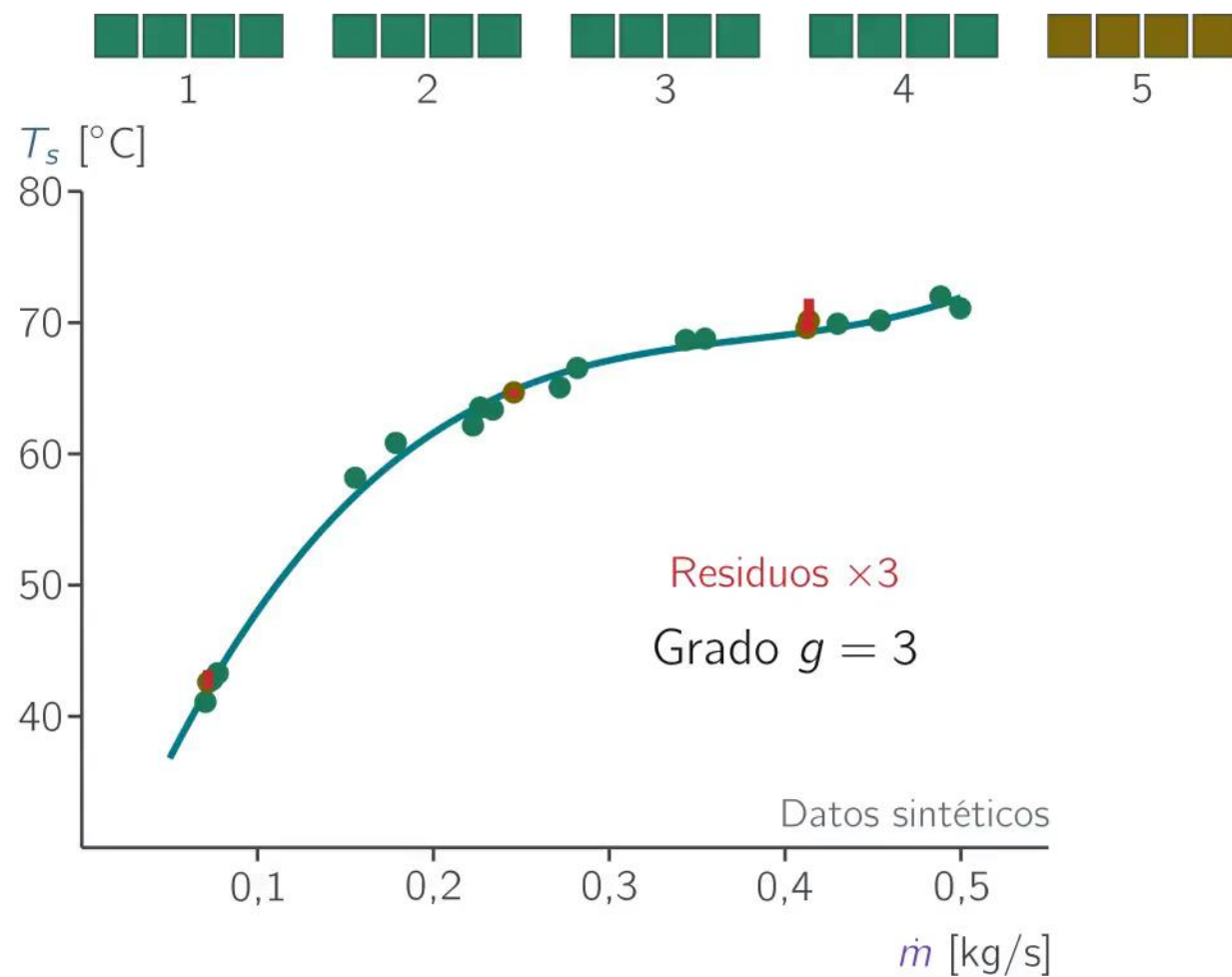
Validación cruzada



- 1 $k = 5$ pliegues (folds) al azar
- 2 Entrenar con $k - 1$, validar con 1
- 3 Rotar el pliegue de validación

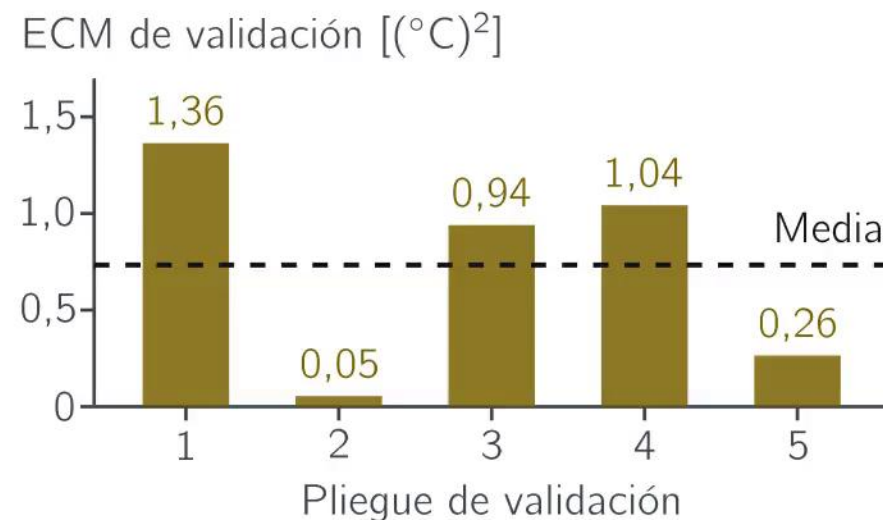


Validación cruzada

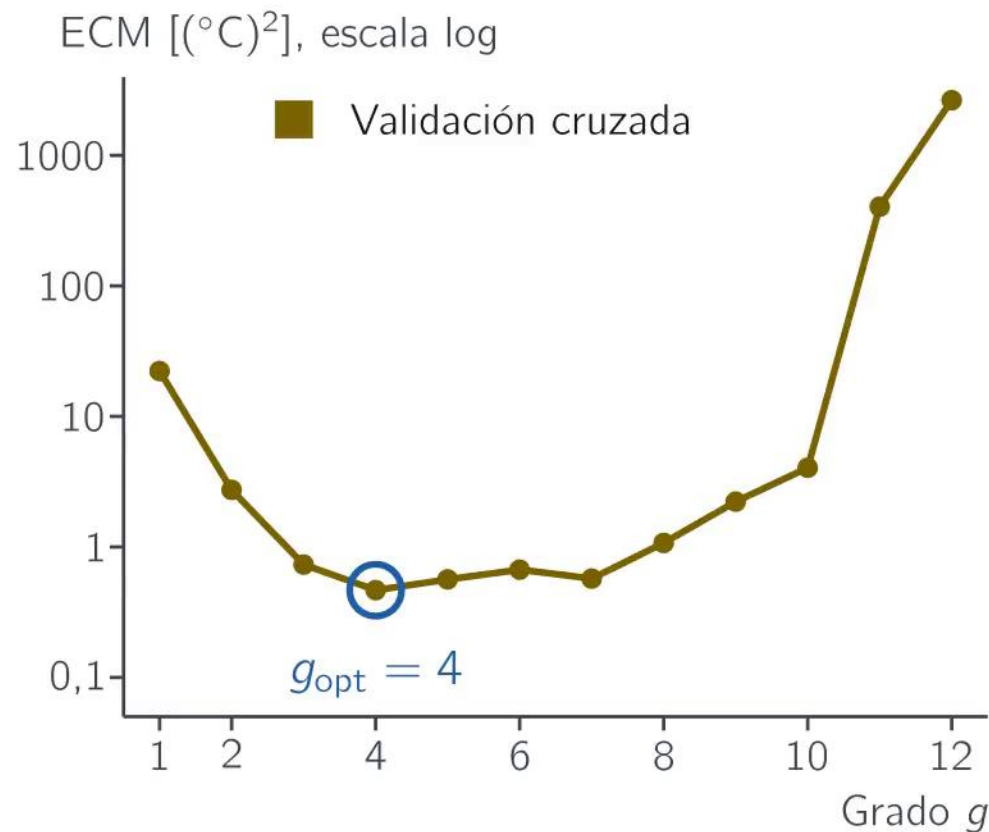
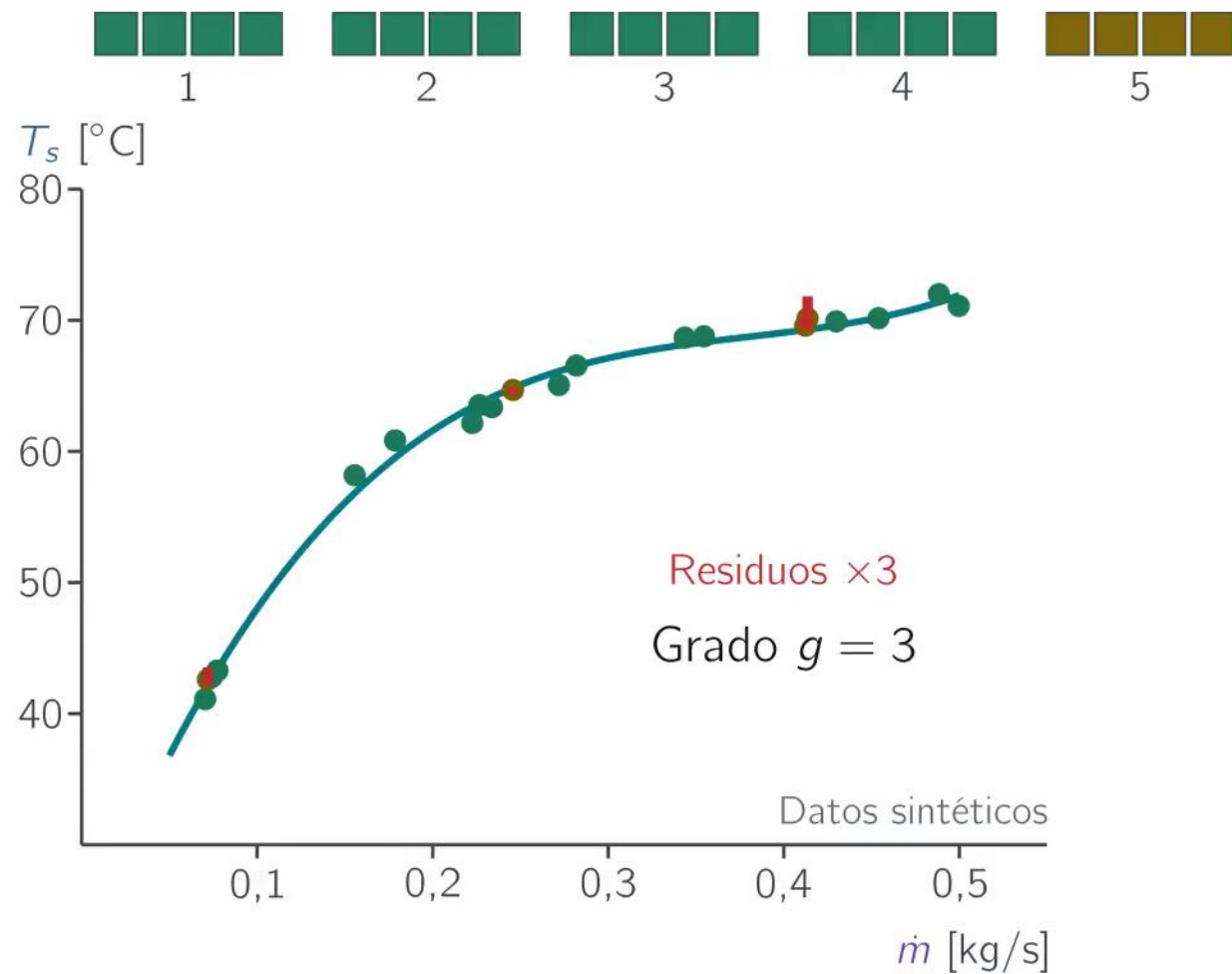


4 Promediar los k errores

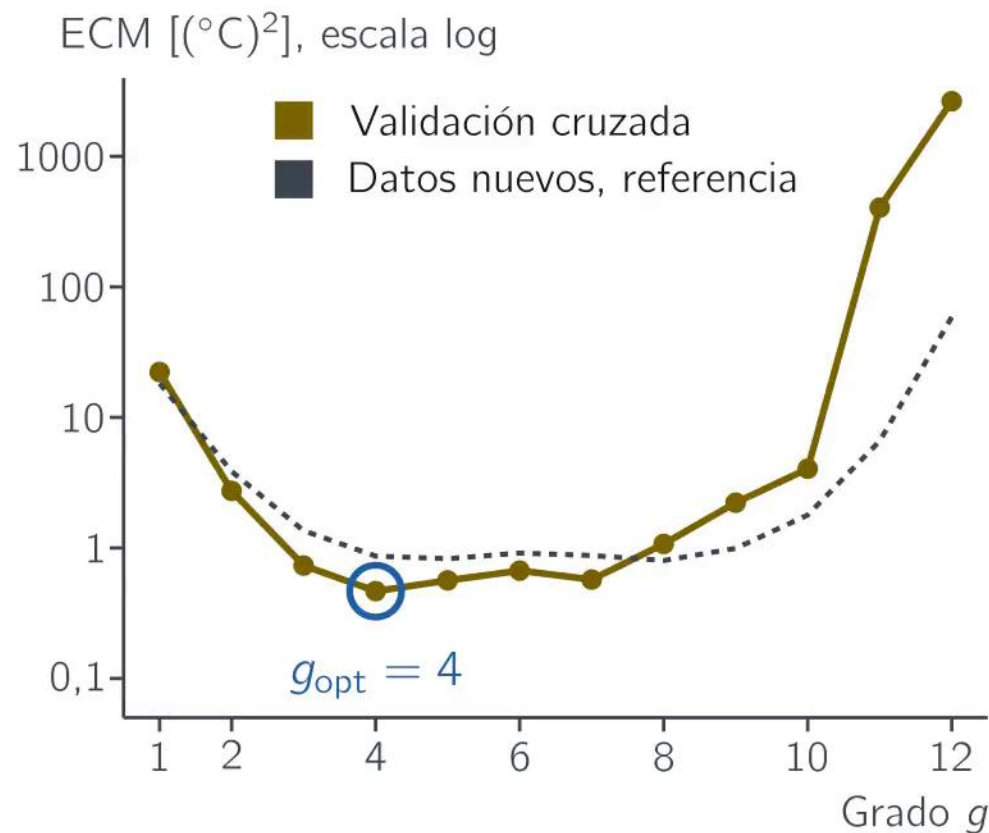
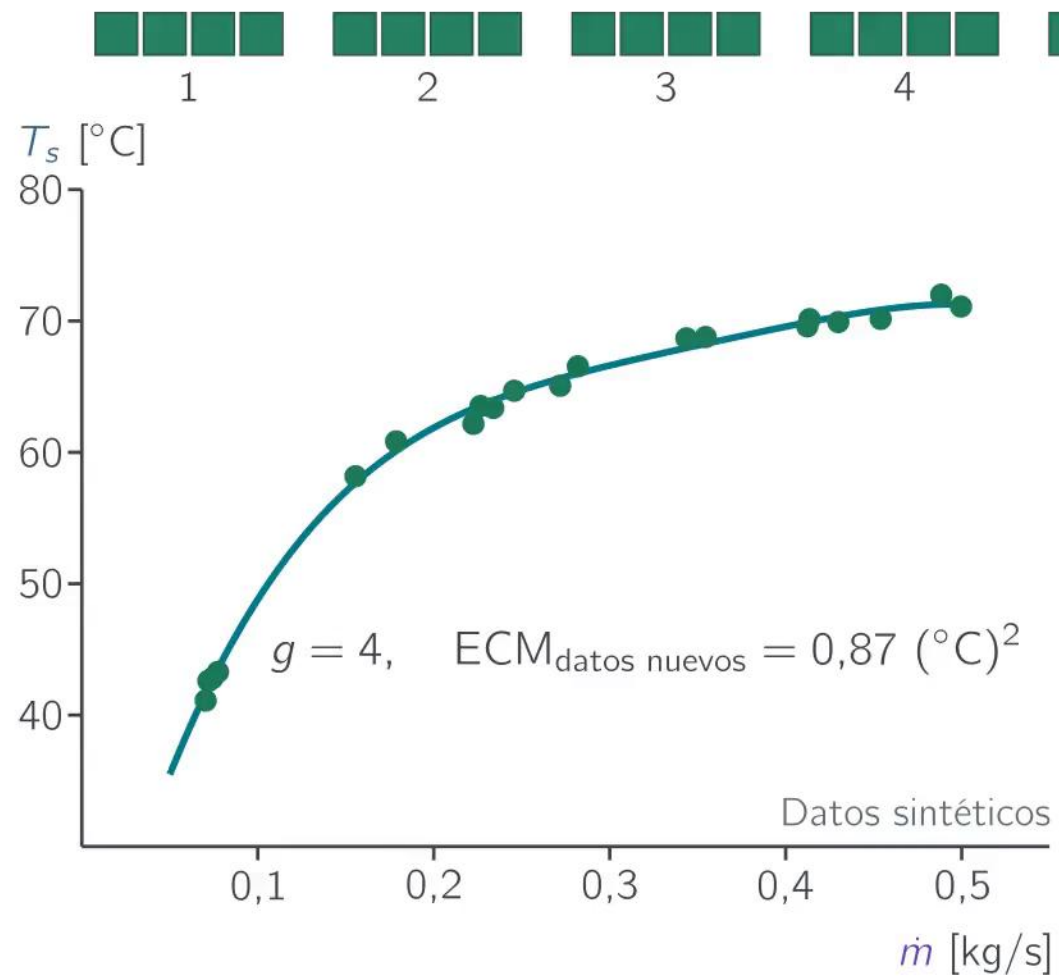
$$CV(g) = \frac{1}{k} \sum_{i=1}^k ECM_i \quad CV(3) = 0,73 \text{ (°C)}^2$$



Validación cruzada

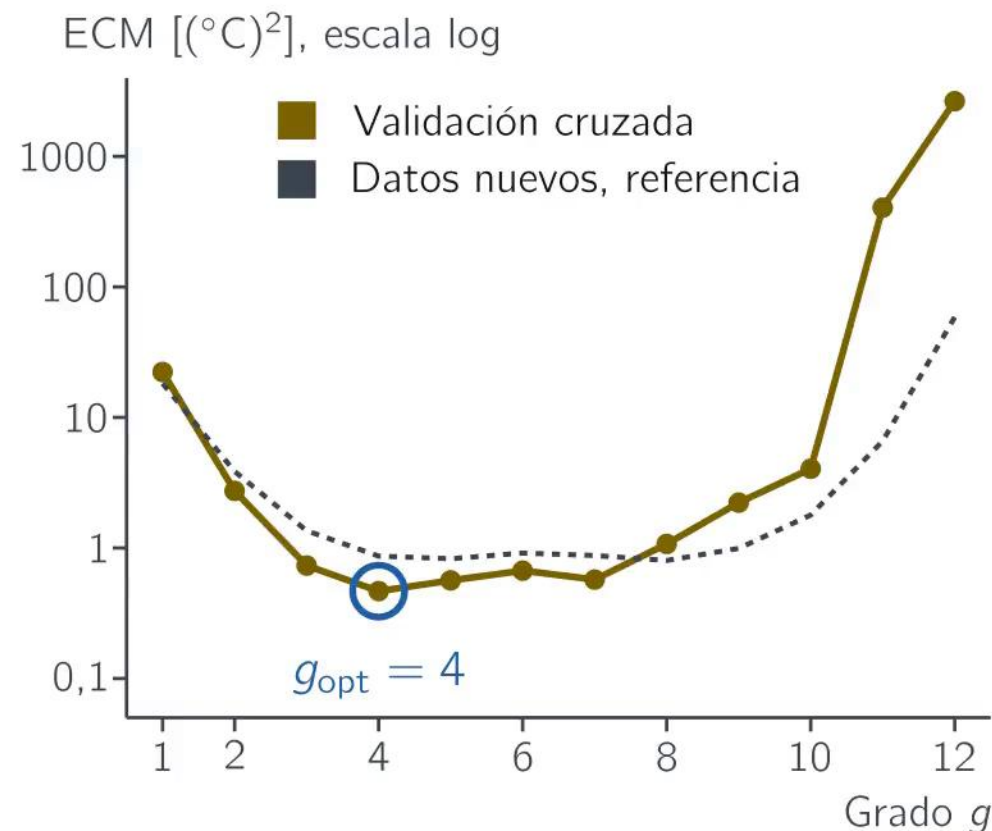


Validación cruzada



Validación cruzada

Pregunta de control. En un caso real guardamos parte de las mediciones como conjunto de prueba. ¿Por qué no elegir el grado con ese conjunto?

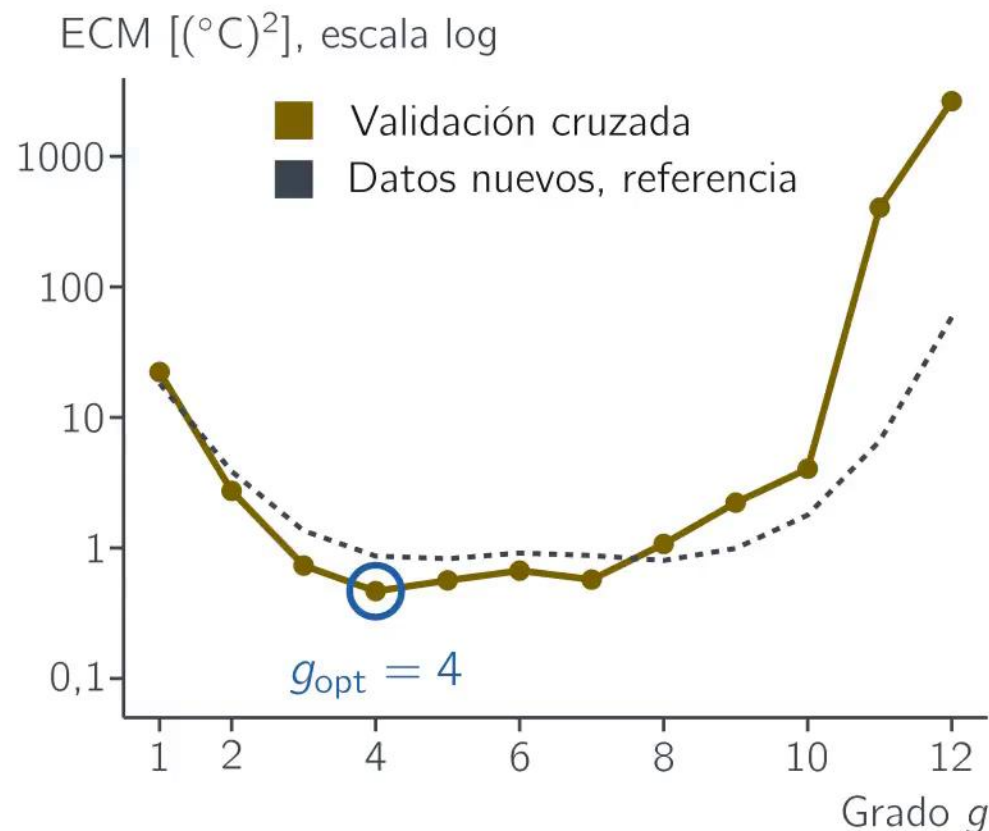


Validación cruzada

Pregunta de control. En un caso real guardamos parte de las mediciones como conjunto de prueba. ¿Por qué no elegir el grado con ese conjunto?

Prueba usada para elegir, error optimista

Prueba, una sola vez al final



Implementación de la validación cruzada

```
def k_fold_error(z, T, grado, k=5):  
    rng = np.random.default_rng(0)  
    folds = np.array_split(rng.permutation(len(z)), k)  
    errores = []  
    for i in range(k):  
        val = folds[i]  
        train = np.hstack(folds[:i] + folds[i+1:])  
        coef = np.polyfit(z[train], T[train], grado)  
        pred = np.polyval(coef, z[val])  
        errores.append(np.mean((pred - T[val])**2))  
    return np.mean(errores)
```

rng.permutation(20)

folds[0]	4	19	6	2
folds[1]	13	16	3	11
folds[2]	10	8	0	12
folds[3]	7	5	18	17
folds[4]	14	9	1	15

Implementación de la validación cruzada

```
def k_fold_error(z, T, grado, k=5):  
    rng = np.random.default_rng(0)  
    folds = np.array_split(rng.permutation(len(z)), k)  
    errores = []  
    for i in range(k):  
        val = folds[i]  
        train = np.hstack(folds[:i] + folds[i+1:])  
        coef = np.polyfit(z[train], T[train], grado)  
        pred = np.polyval(coef, z[val])  
        errores.append(np.mean((pred - T[val])**2))  
    return np.mean(errores)
```

	rng.permutation(20)				errores
folds[0]	4	19	6	2	1,36
folds[1]	13	16	3	11	0,05
folds[2]	10	8	0	12	0,94
folds[3]	7	5	18	17	1,04
folds[4]	14	9	1	15	0,26

grado = 3

Implementación de la validación cruzada

```
def k_fold_error(z, T, grado, k=5):  
    rng = np.random.default_rng(0)  
    folds = np.array_split(rng.permutation(len(z)), k)  
    errores = []  
    for i in range(k):  
        val = folds[i]  
        train = np.hstack(folds[:i] + folds[i+1:])  
        coef = np.polyfit(z[train], T[train], grado)  
        pred = np.polyval(coef, z[val])  
        errores.append(np.mean((pred - T[val])**2))  
    return np.mean(errores)
```

```
grados = range(1, 13)  
err_cv = [k_fold_error(z, T, g) for g in grados]  
g_opt = grados[np.argmin(err_cv)]
```

	rng.permutation(20)				errores
folds[0]	4	19	6	2	1,36
folds[1]	13	16	3	11	0,05
folds[2]	10	8	0	12	0,94
folds[3]	7	5	18	17	1,04
folds[4]	14	9	1	15	0,26

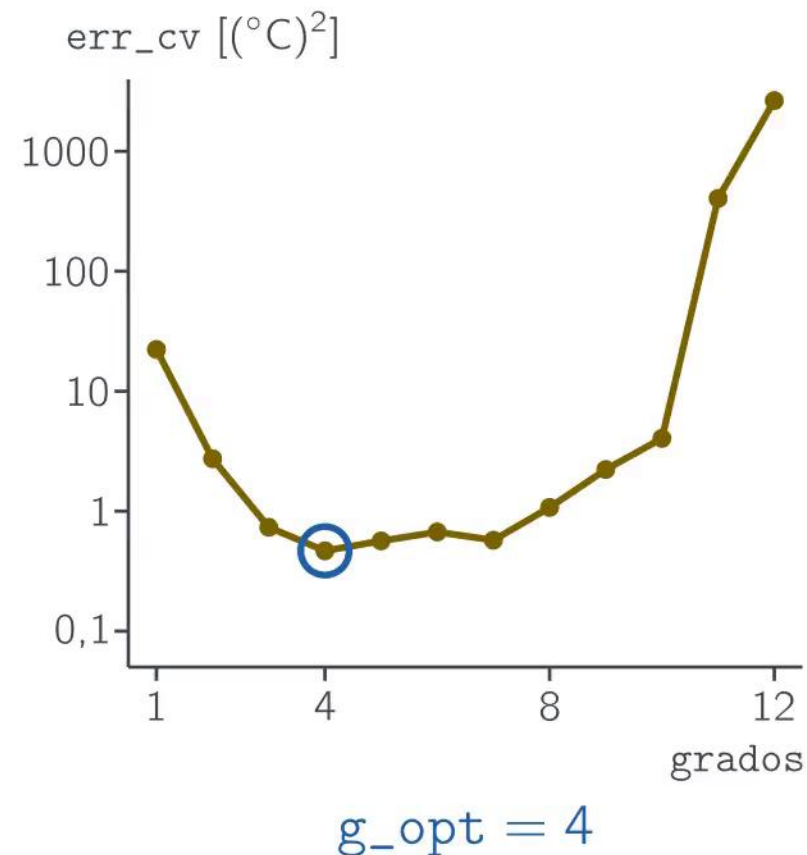
grado = 3

$\text{np.mean(errores)} = 0,73$

Implementación de la validación cruzada

```
def k_fold_error(z, T, grado, k=5):  
    rng = np.random.default_rng(0)  
    folds = np.array_split(rng.permutation(len(z)), k)  
    errores = []  
    for i in range(k):  
        val = folds[i]  
        train = np.hstack(folds[:i] + folds[i+1:])  
        coef = np.polyfit(z[train], T[train], grado)  
        pred = np.polyval(coef, z[val])  
        errores.append(np.mean((pred - T[val])**2))  
    return np.mean(errores)
```

```
grados = range(1, 13)  
err_cv = [k_fold_error(z, T, g) for g in grados]  
g_opt = grados[np.argmin(err_cv)]
```

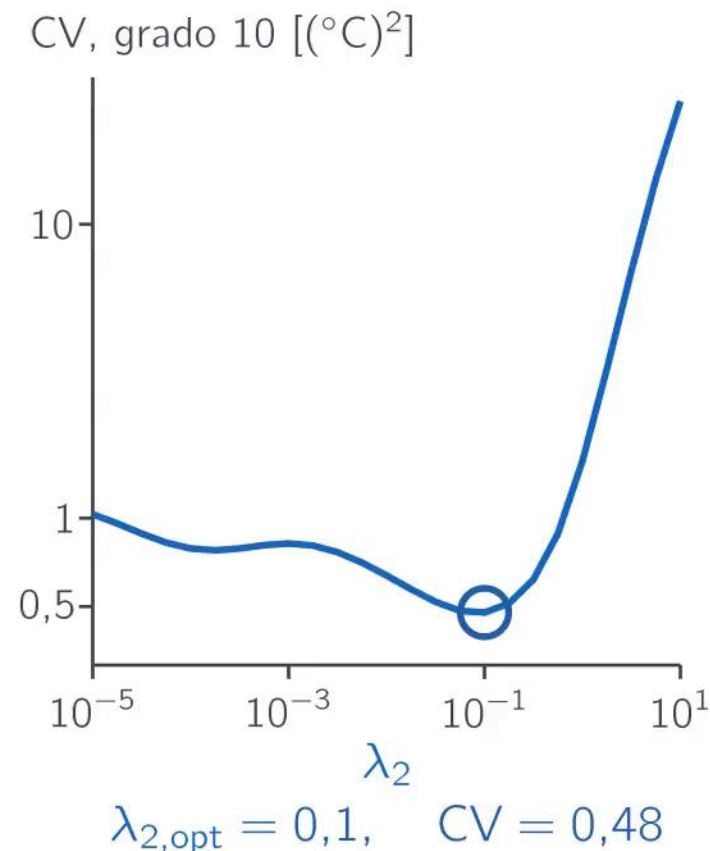


Implementación de la validación cruzada

```
def k_fold_ridge(z, T, lam, k=5):  
    rng = np.random.default_rng(0)  
    folds = np.array_split(rng.permutation(len(z)), k)  
    errores = []  
    for i in range(k):  
        val = folds[i]  
        train = np.hstack(folds[:i] + folds[i+1:])  
        A = np.vander(z[train], 11, increasing=True)  
        x = np.linalg.solve(A.T @ A + lam * D, A.T @ T[train])  
        pred = np.vander(z[val], 11, increasing=True) @ x  
        errores.append(np.mean((pred - T[val])**2))  
    return np.mean(errores)
```

$D = \text{np.diag}([0.0] + [1.0] * 10)$

x_0 sin penalización



Contenidos

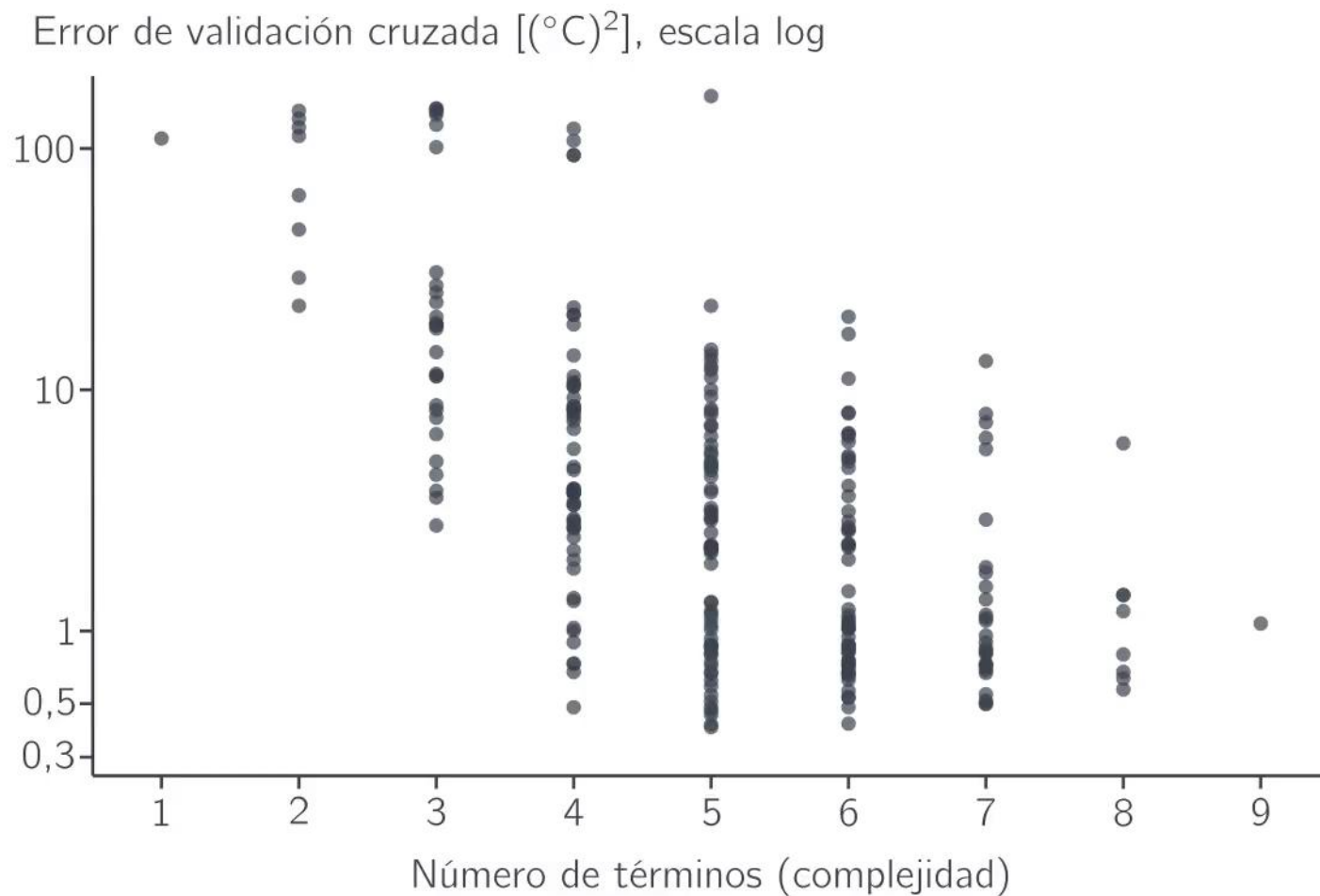
1 Generalización

2 Pareto

3 Clasificación

4 PCA

Frontera de Pareto entre error y complejidad

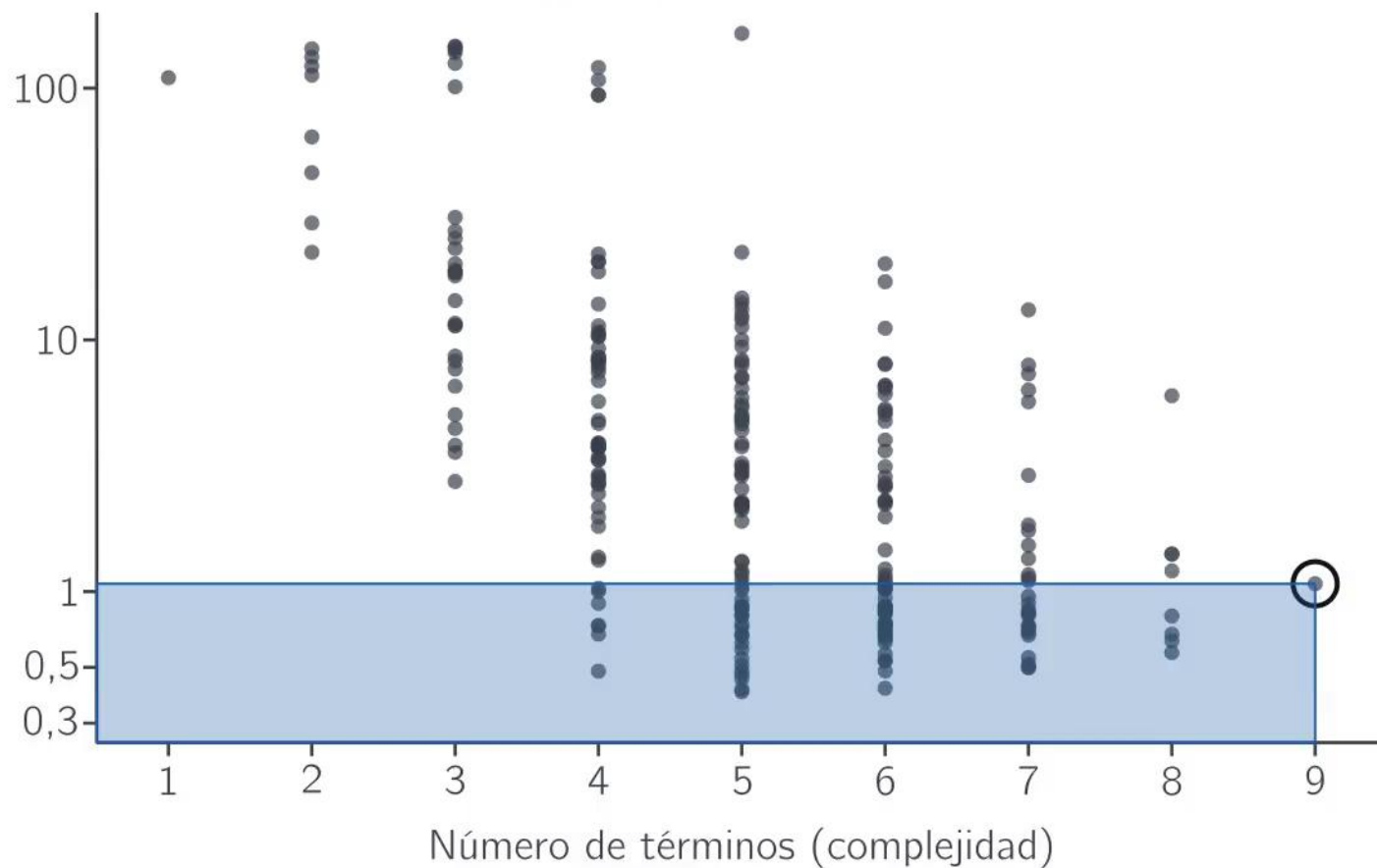


256 modelos

Términos $z, \dots, z^8$ más constante

Frontera de Pareto entre error y complejidad

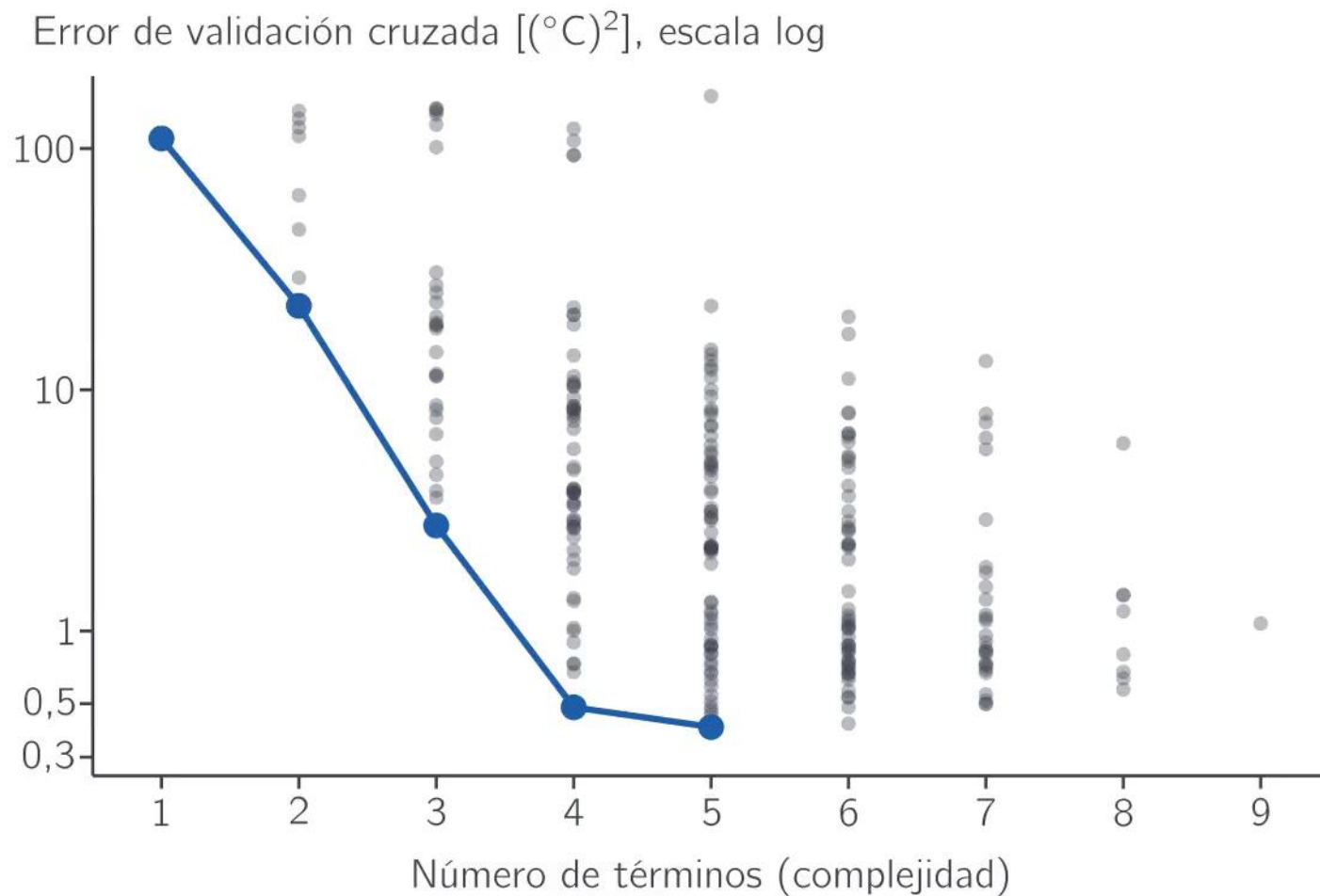
Error de validación cruzada $[(^{\circ}\text{C})^2]$, escala log



Dominado, otro con igual o menos términos y menor error

Grado 8, 9 términos, $CV = 1,07$

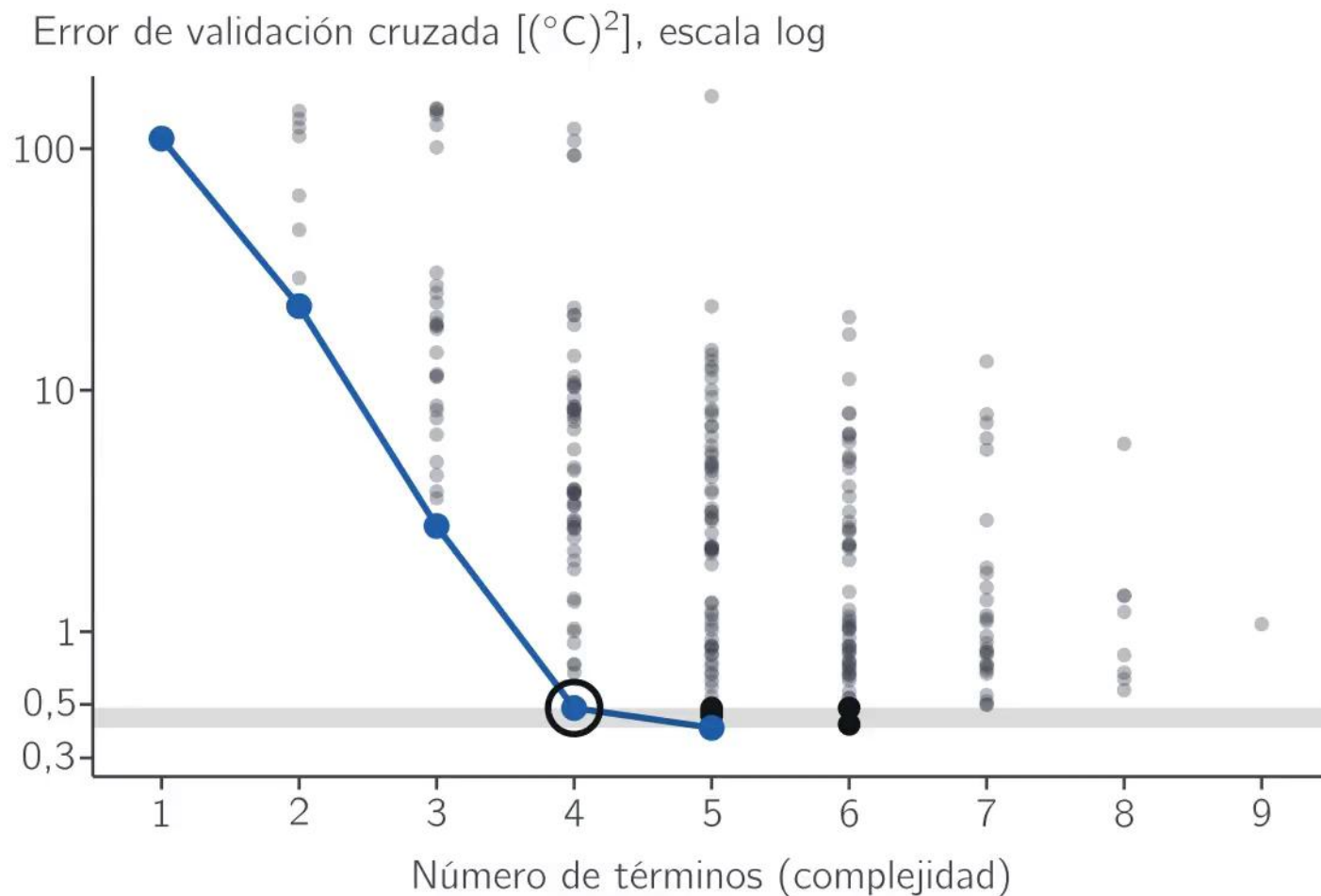
Frontera de Pareto entre error y complejidad



Frontera de Pareto, modelos no dominados

1 término	110,05
2 términos	22,28
3 términos	2,74
4 términos	0,48
5 términos	0,40

Frontera de Pareto entre error y complejidad



Navaja de Occam, con error similar, el modelo más simple

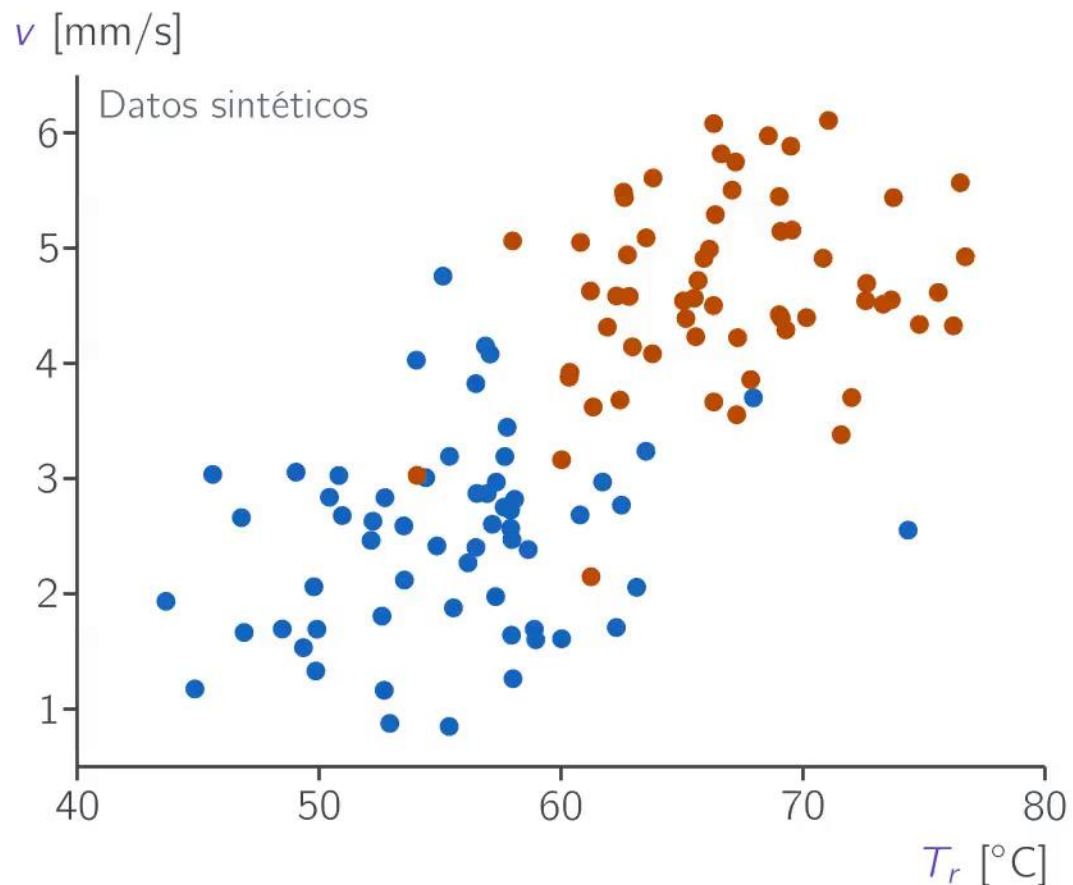
Franja de 1 error estándar, 8 modelos

$$\hat{T}_s = x_0 + x_1 z + x_4 z^4 + x_5 z^5$$

Contenidos

- 1 Generalización
- 2 Pareto
- 3 Clasificación
- 4 PCA

Regresión logística



Rodamiento de bomba, 120 revisiones

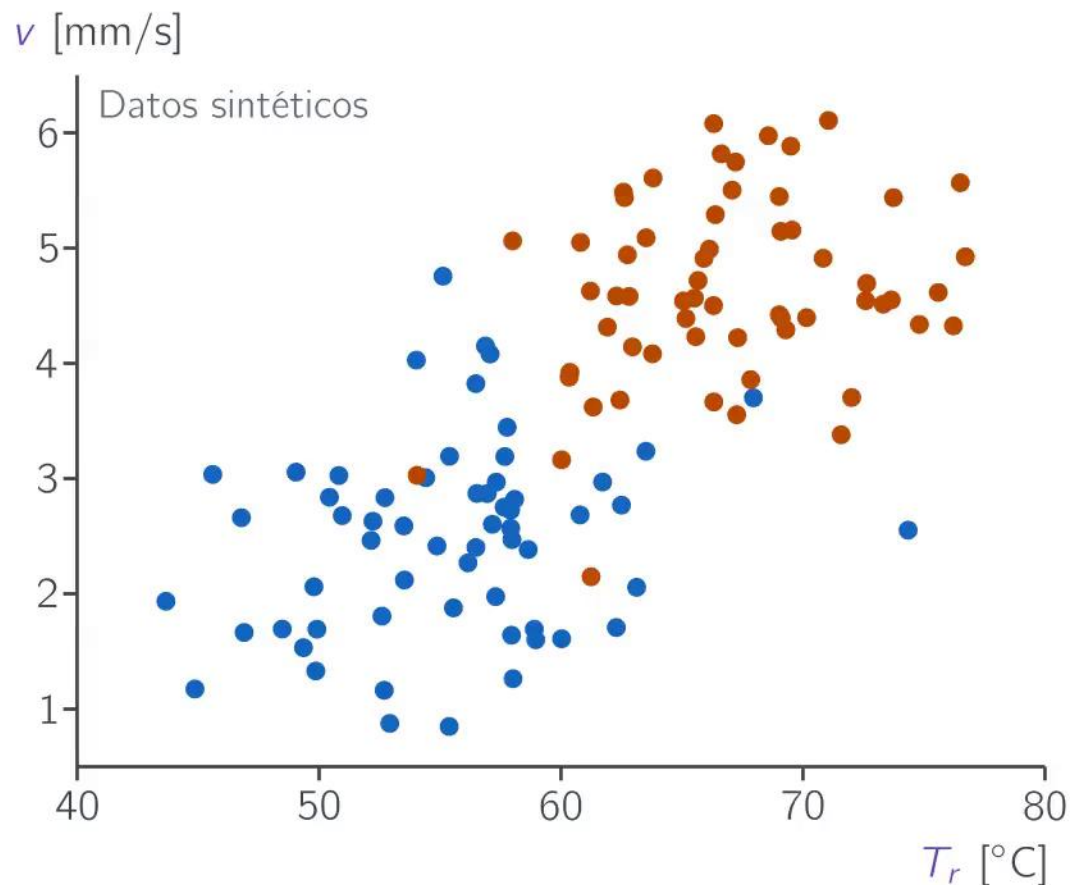
■ $y = 0$, sin falla

■ $y = 1$, falla

Probabilidad de falla de una revisión nueva

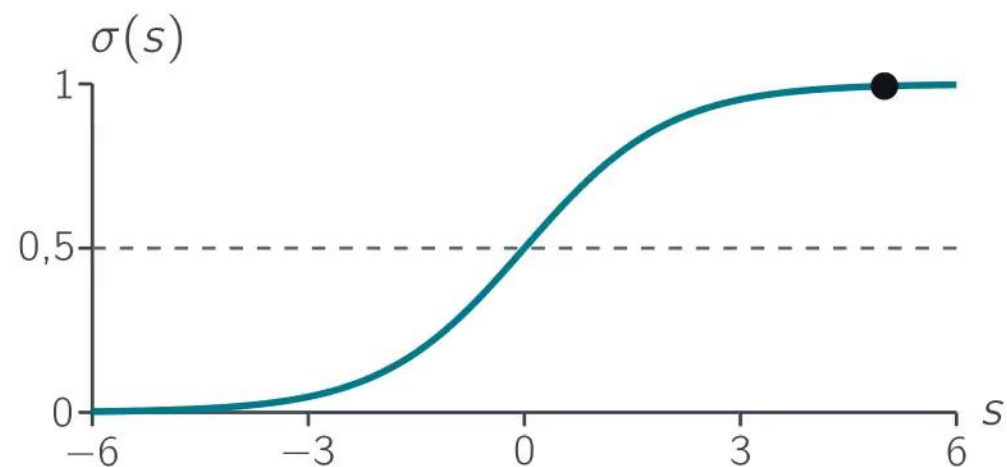
$$x_1 = \frac{T_r - 60}{10} \quad x_2 = \frac{v - 3,5}{1,5}$$

Regresión logística



$$s = w_1 x_1 + w_2 x_2 + w_0$$

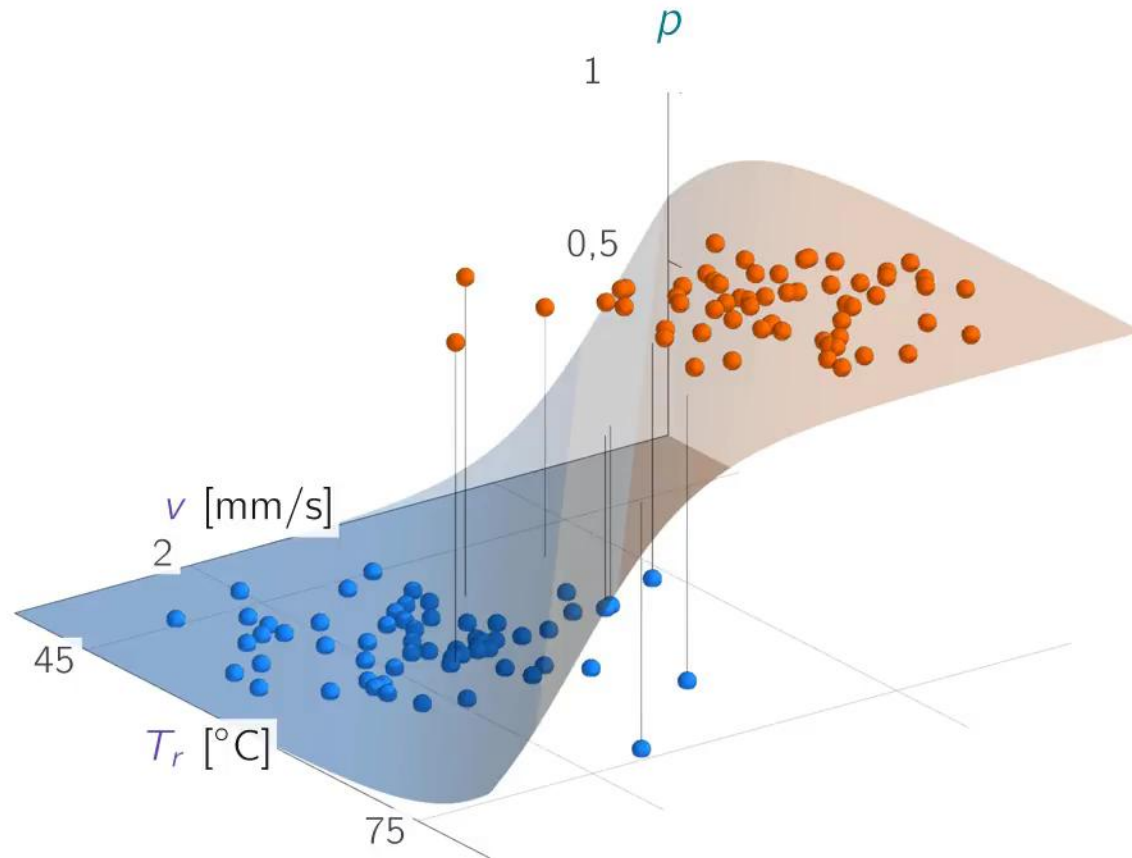
$$p = \sigma(s) = \frac{1}{1 + e^{-s}} \in (0, 1)$$



$Xw + w_0$, el Ax de ayer

w_0 término constante, b en muchos códigos

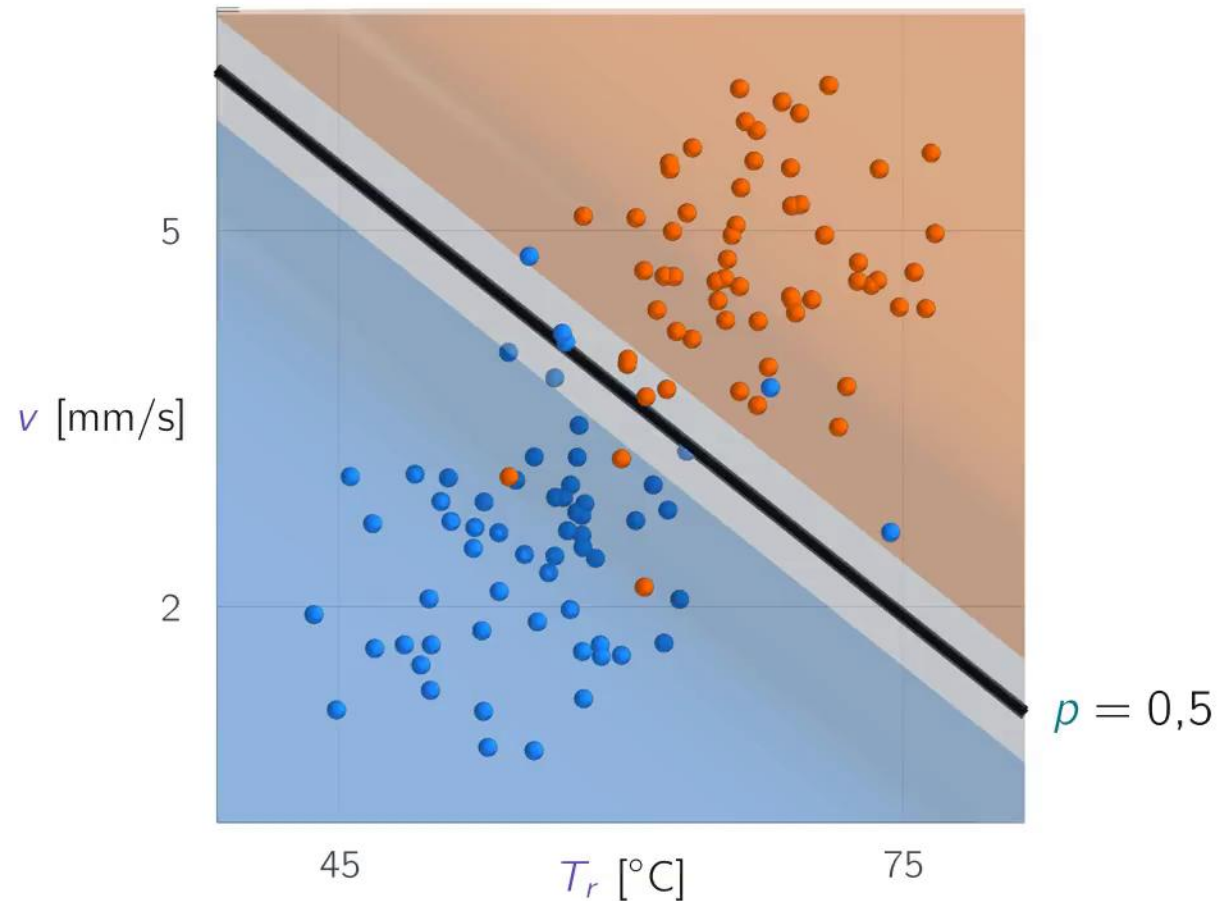
Regresión logística



Altura p , pesos tras 500 iteraciones

Falla a altura 1, sin falla a altura 0

Regresión logística

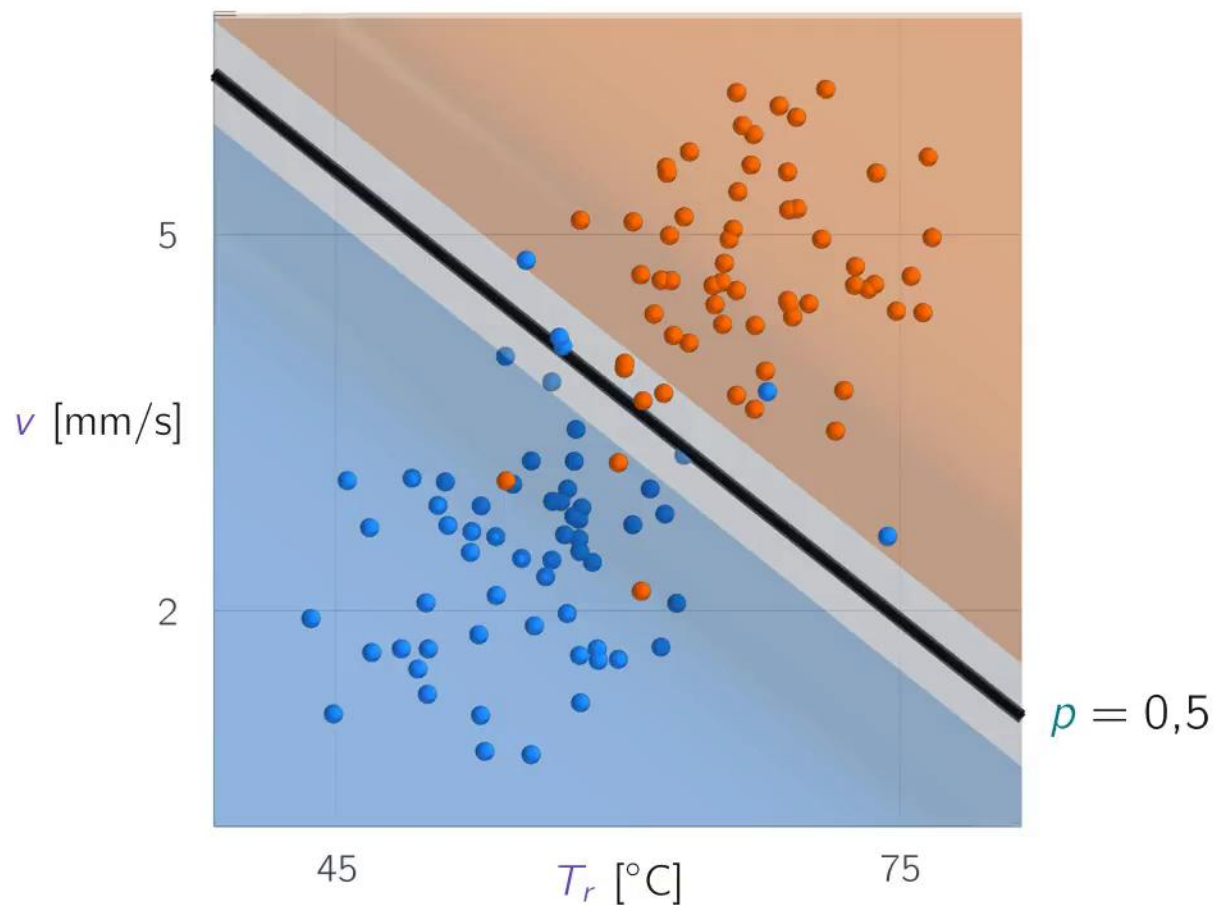


$$p = 0,5 \iff e^{-s} = 1 \iff s = 0$$

$$2,06 x_1 + 2,58 x_2 - 0,35 = 0$$

Frontera de decisión, una recta

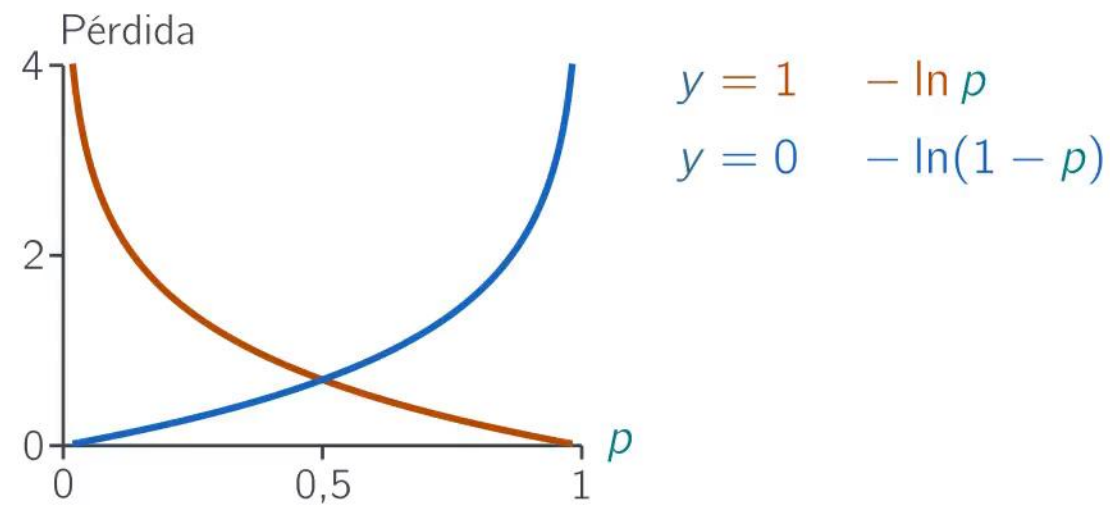
Regresión logística



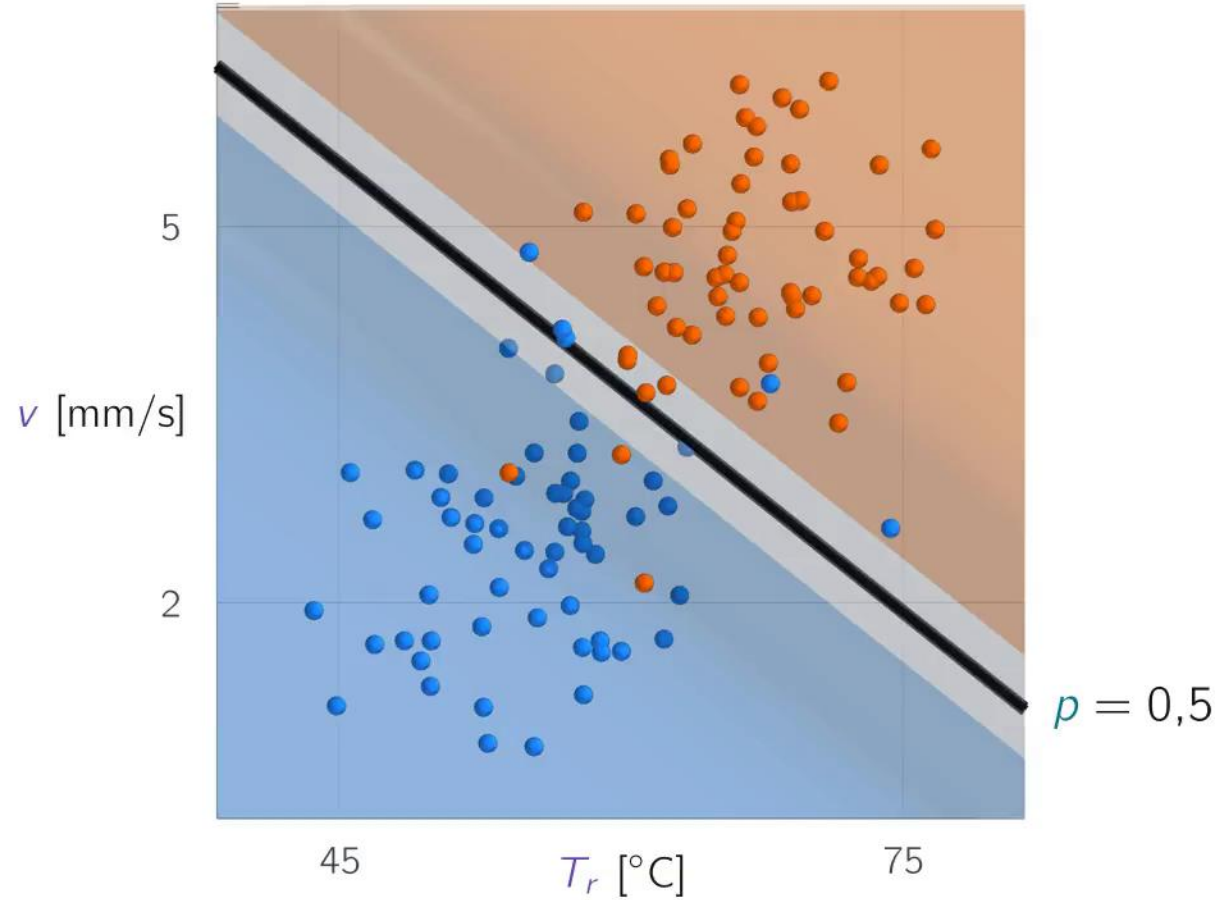
$$L(w) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]$$

Entropía cruzada, convexa en (w, w_0)

Error cuadrático con sigmoide, no convexo



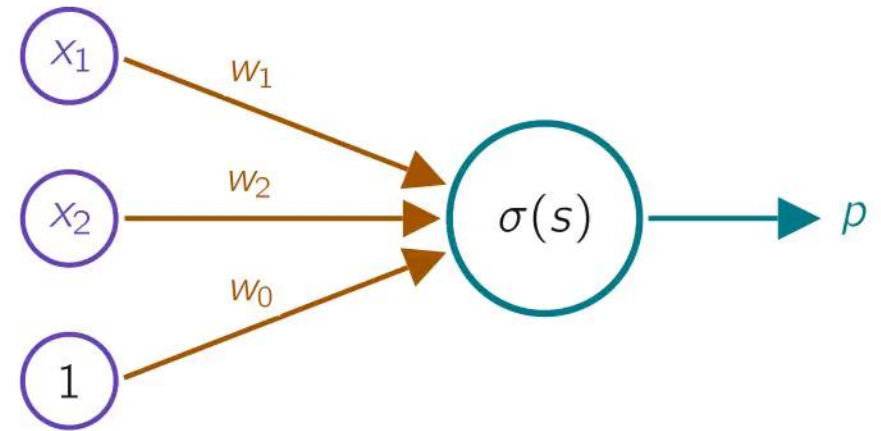
Regresión logística



$$L(w) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]$$

Una neurona artificial

Clase 6, muchas neuronas en capas



Implementación de la regresión logística

```
def sigmoid(s):  
    return 1.0 / (1.0 + np.exp(-s))
```

```
w, w0 = np.zeros(2), 0.0
```

```
lr = 0.1
```

```
for _ in range(500):
```

```
    p = sigmoid(X @ w + w0)
```

```
    grad_w = X.T @ (p - y) / len(y)
```

```
    grad_w0 = np.mean(p - y)
```

```
    w -= lr * grad_w
```

```
    w0 -= lr * grad_w0
```

$$w = \mathbf{0}, w_0 = 0 \Rightarrow p_i = \sigma(0) = 0,5$$

$$L = -\ln 0,5 = \ln 2 = 0,693$$

$$w_0 \rightarrow w0, \quad y \rightarrow y, \quad \alpha \rightarrow lr$$

Implementación de la regresión logística

```
def sigmoid(s):  
    return 1.0 / (1.0 + np.exp(-s))  
  
w, w0 = np.zeros(2), 0.0  
lr = 0.1  
for _ in range(500):  
    p = sigmoid(X @ w + w0)  
    grad_w = X.T @ (p - y) / len(y)  
    grad_w0 = np.mean(p - y)  
    w -= lr * grad_w  
    w0 -= lr * grad_w0
```

$$\sigma'(s) = \frac{e^{-s}}{(1 + e^{-s})^2} = \sigma(s) (1 - \sigma(s))$$

Implementación de la regresión logística

```
def sigmoid(s):  
    return 1.0 / (1.0 + np.exp(-s))  
  
w, w0 = np.zeros(2), 0.0  
lr = 0.1  
for _ in range(500):  
    p = sigmoid(X @ w + w0)  
    grad_w = X.T @ (p - y) / len(y)  
    grad_w0 = np.mean(p - y)  
    w -= lr * grad_w  
    w0 -= lr * grad_w0
```

$$\sigma'(s) = \frac{e^{-s}}{(1 + e^{-s})^2} = \sigma(s) (1 - \sigma(s))$$

$$\ell_i = -[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)], \quad p_i = \sigma(s_i)$$

$$\frac{\partial \ell_i}{\partial s_i} = -\left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i}\right] p_i (1 - p_i) = p_i - y_i$$

Implementación de la regresión logística

```
def sigmoid(s):  
    return 1.0 / (1.0 + np.exp(-s))  
  
w, w0 = np.zeros(2), 0.0  
lr = 0.1  
for _ in range(500):  
    p = sigmoid(X @ w + w0)  
    grad_w = X.T @ (p - y) / len(y)  
    grad_w0 = np.mean(p - y)  
    w -= lr * grad_w  
    w0 -= lr * grad_w0
```

$$\frac{\partial \ell_i}{\partial s_i} = - \left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right] p_i (1 - p_i) = p_i - y_i$$

$$\nabla_w L = \frac{1}{m} \sum_i (p_i - y_i) \mathbf{x}_i = \frac{1}{m} X^T (p - y)$$

$$\frac{\partial L}{\partial w_0} = \frac{1}{m} \sum_i (p_i - y_i)$$

Mínimos cuadrados, ayer

$$\nabla_x \|Ax - b\|^2 = 2A^T(Ax - b)$$

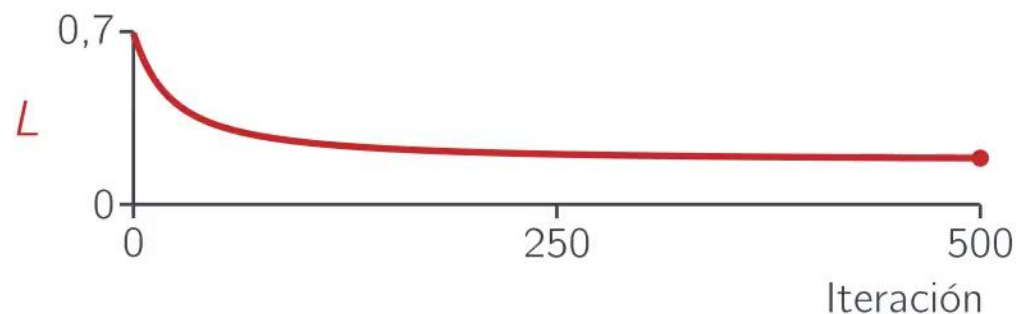
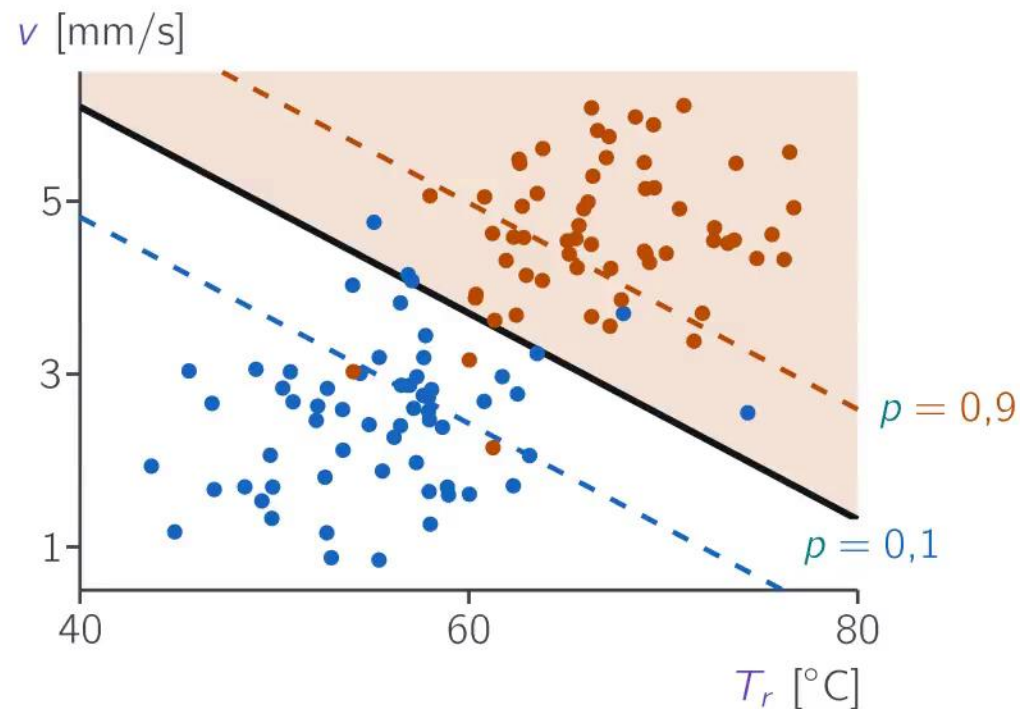
Implementación de la regresión logística

```
def sigmoid(s):  
    return 1.0 / (1.0 + np.exp(-s))  
  
w, w0 = np.zeros(2), 0.0  
lr = 0.1  
for _ in range(500):  
    p = sigmoid(X @ w + w0)  
    grad_w = X.T @ (p - y) / len(y)  
    grad_w0 = np.mean(p - y)  
    w -= lr * grad_w  
    w0 -= lr * grad_w0
```

Iteración 500

$L = 0,187$

Exactitud (entrenamiento) 93,3 %



Implementación de la regresión logística

Tras 500 iteraciones

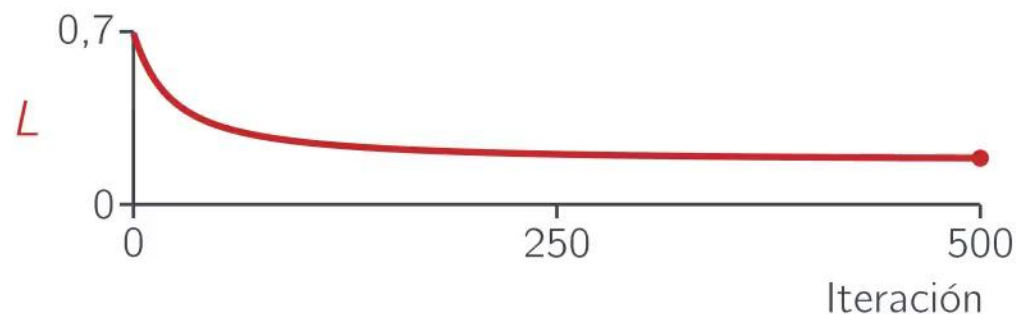
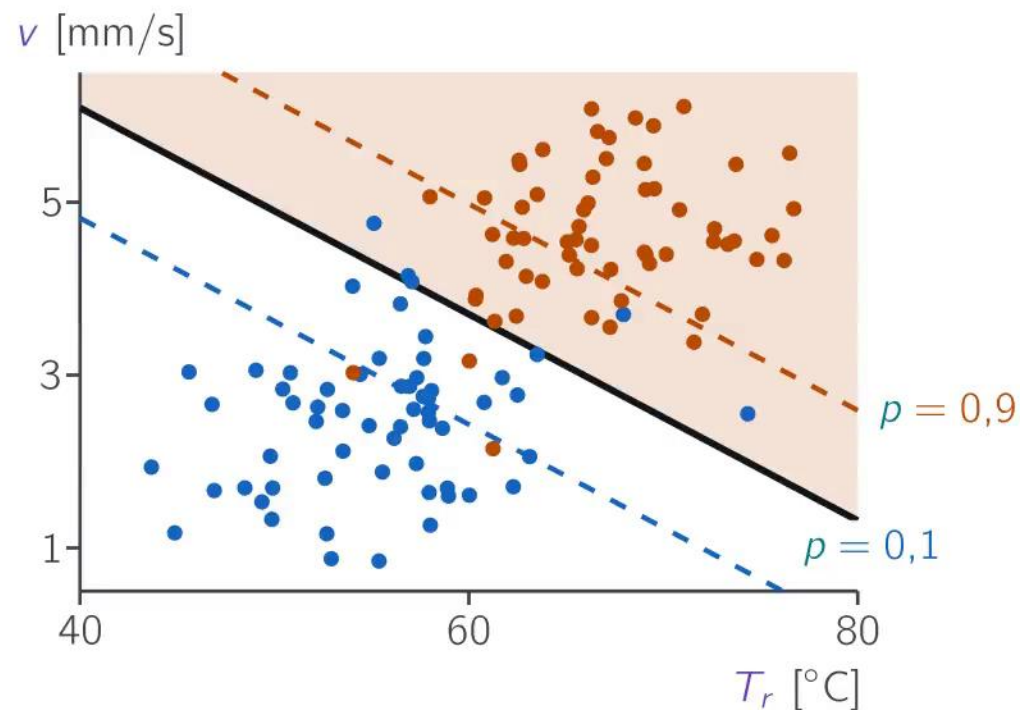
$$w_1 = 2,06 \quad w_2 = 2,58 \quad w_0 = -0,35$$

Mismo descenso de gradiente
de la clase 4, con otra pérdida.

Iteración 500

$$L = 0,187$$

Exactitud (entrenamiento) 93,3 %



Contenidos

- 1 Generalización
- 2 Pareto
- 3 Clasificación
- 4 PCA

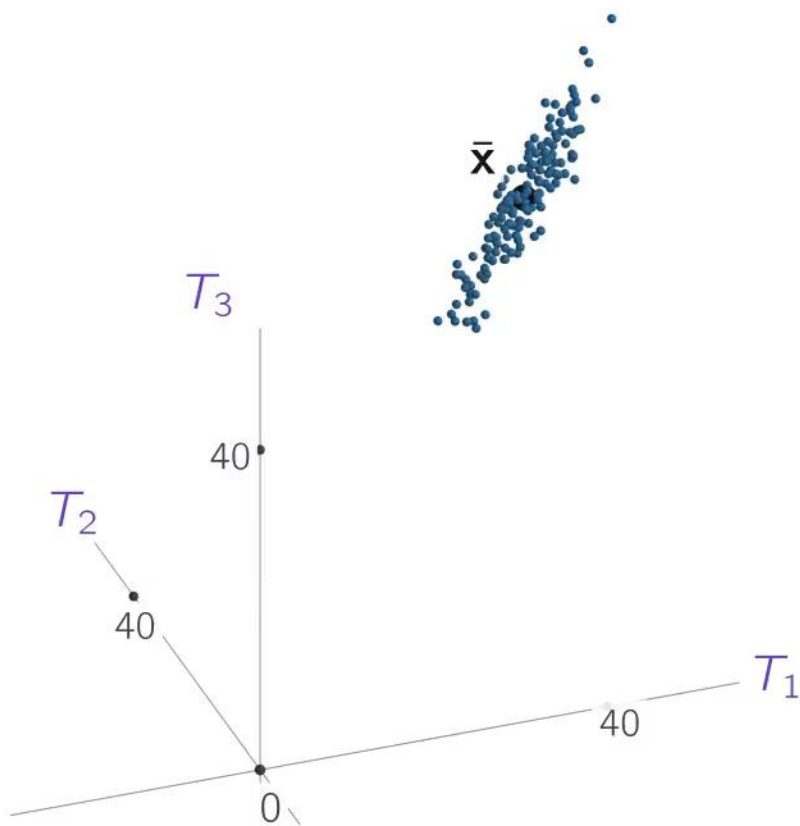
Análisis de componentes principales

3 termocuplas en una placa, 150 lecturas

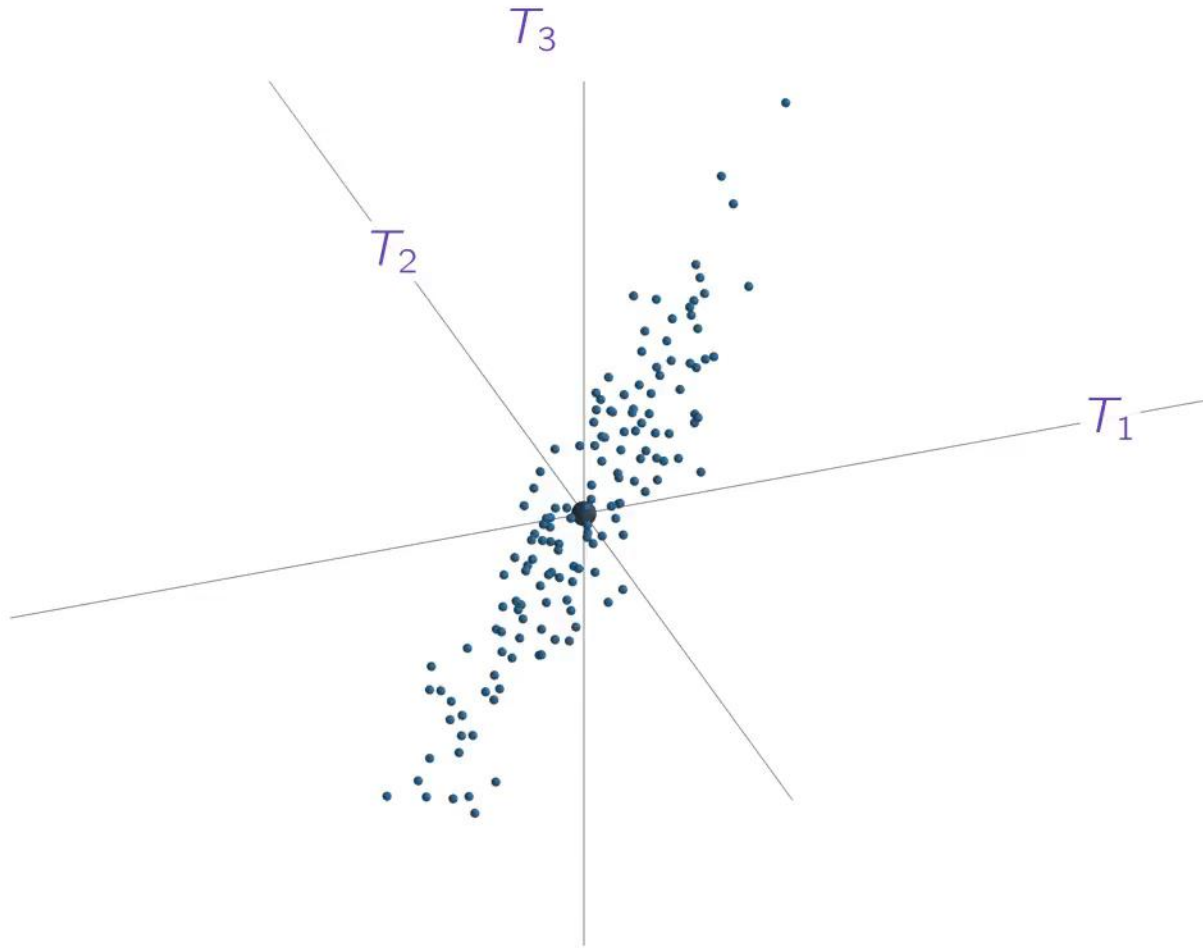
$X \in \mathbb{R}^{150 \times 3}$, fila $i = (T_1, T_2, T_3)_i$

$$\bar{\mathbf{x}} = \begin{pmatrix} 46,0 \\ 43,0 \\ 39,1 \end{pmatrix} ^\circ\text{C}$$

Datos sintéticos



Análisis de componentes principales



3 termocuplas en una placa, 150 lecturas

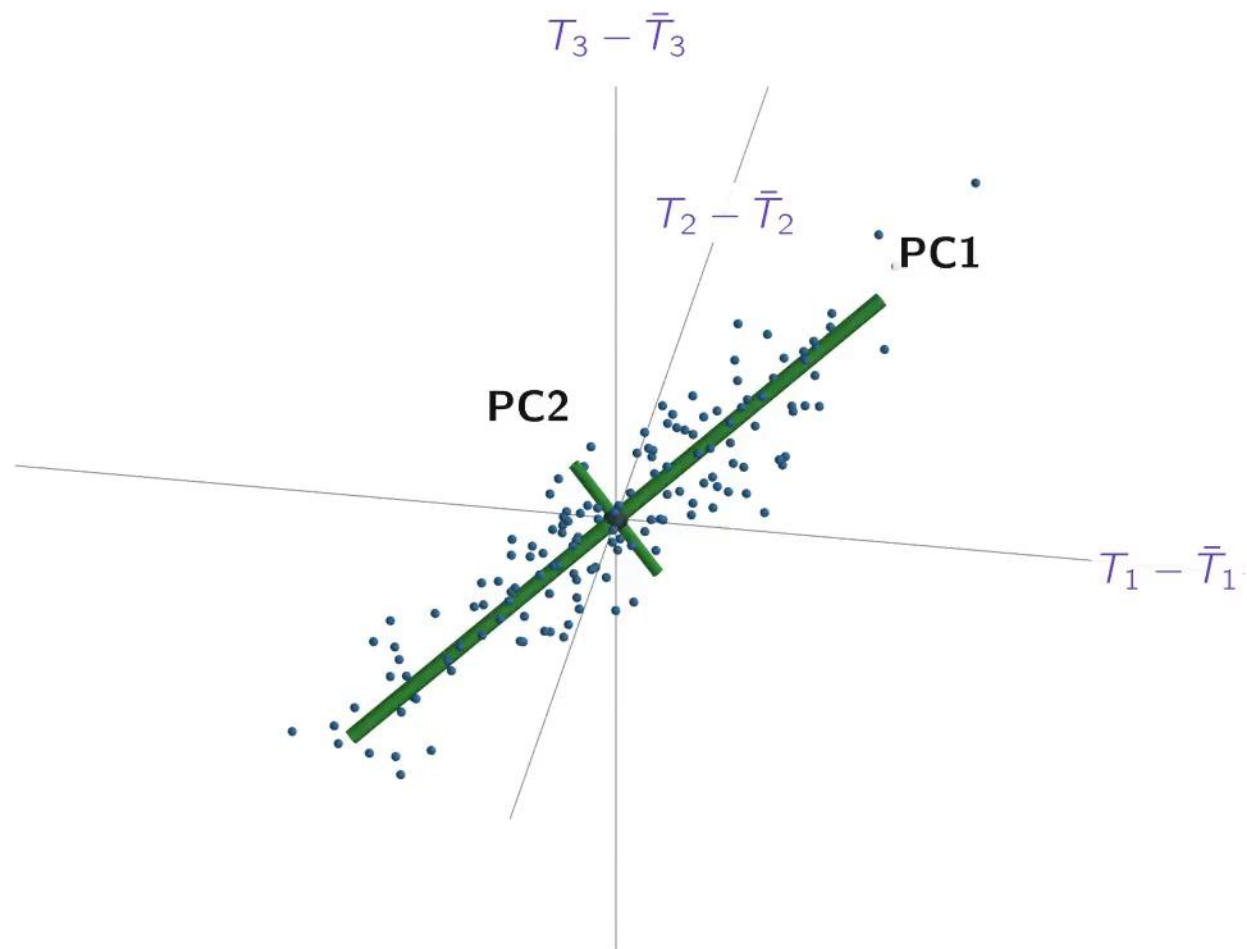
$X \in \mathbb{R}^{150 \times 3}$, fila $i = (T_1, T_2, T_3)_i$

$$\bar{\mathbf{x}} = \begin{pmatrix} 46,0 \\ 43,0 \\ 39,1 \end{pmatrix} ^\circ\text{C}$$

$$X_c = X - \mathbf{1} \bar{\mathbf{x}}^T$$

Promedio cero en cada columna

Análisis de componentes principales



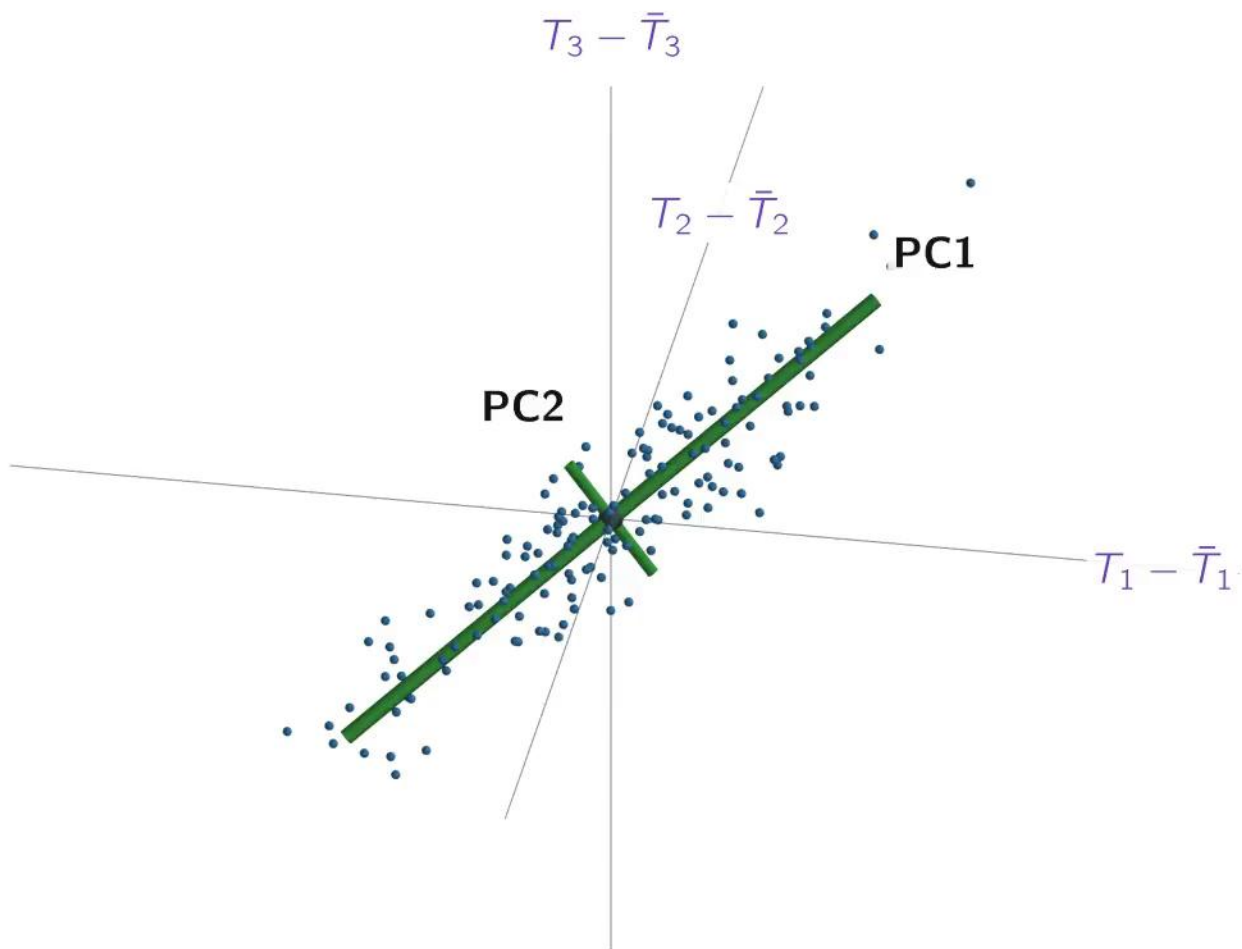
$$X_c = U \Sigma V^T$$

Filas de V^T , direcciones principales

$$\sigma_1 = 100,2 \quad \sigma_2 = 20,3 \quad \sigma_3 = 5,9$$

Pregunta de control. ¿Cuántas direcciones hacen falta?

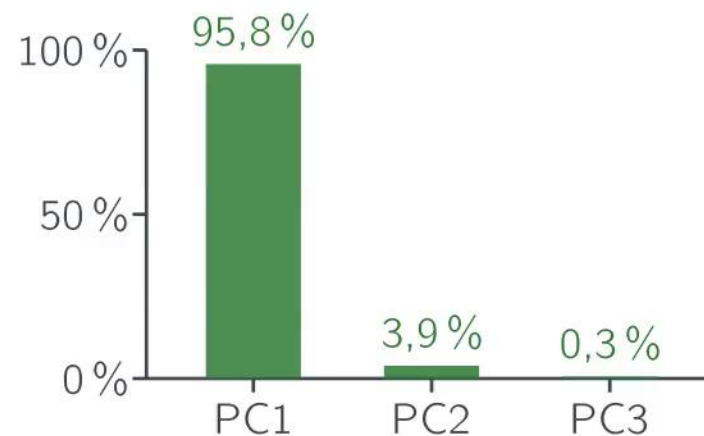
Análisis de componentes principales



$$X_c = U \Sigma V^T$$

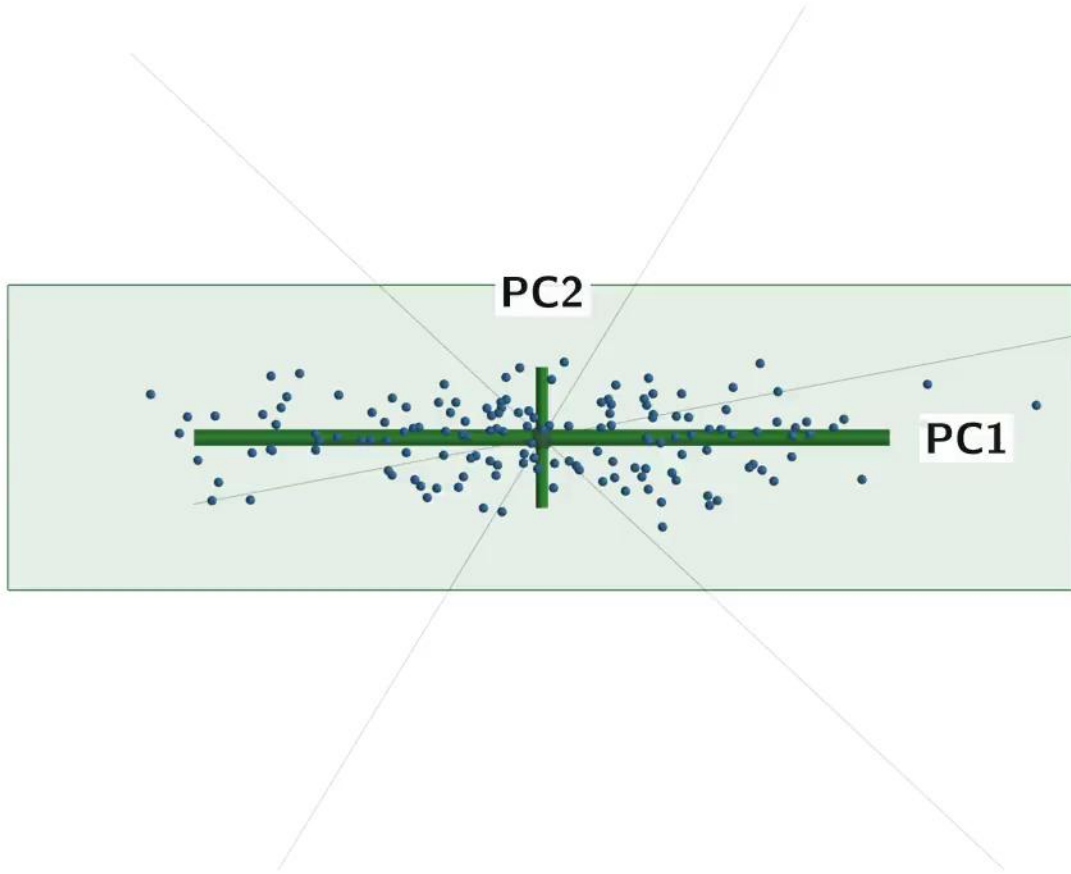
Filas de V^T , direcciones principales

$$\sigma_1 = 100,2 \quad \sigma_2 = 20,3 \quad \sigma_3 = 5,9$$



$$\frac{\sigma_i^2}{\sum_j \sigma_j^2}$$

Análisis de componentes principales

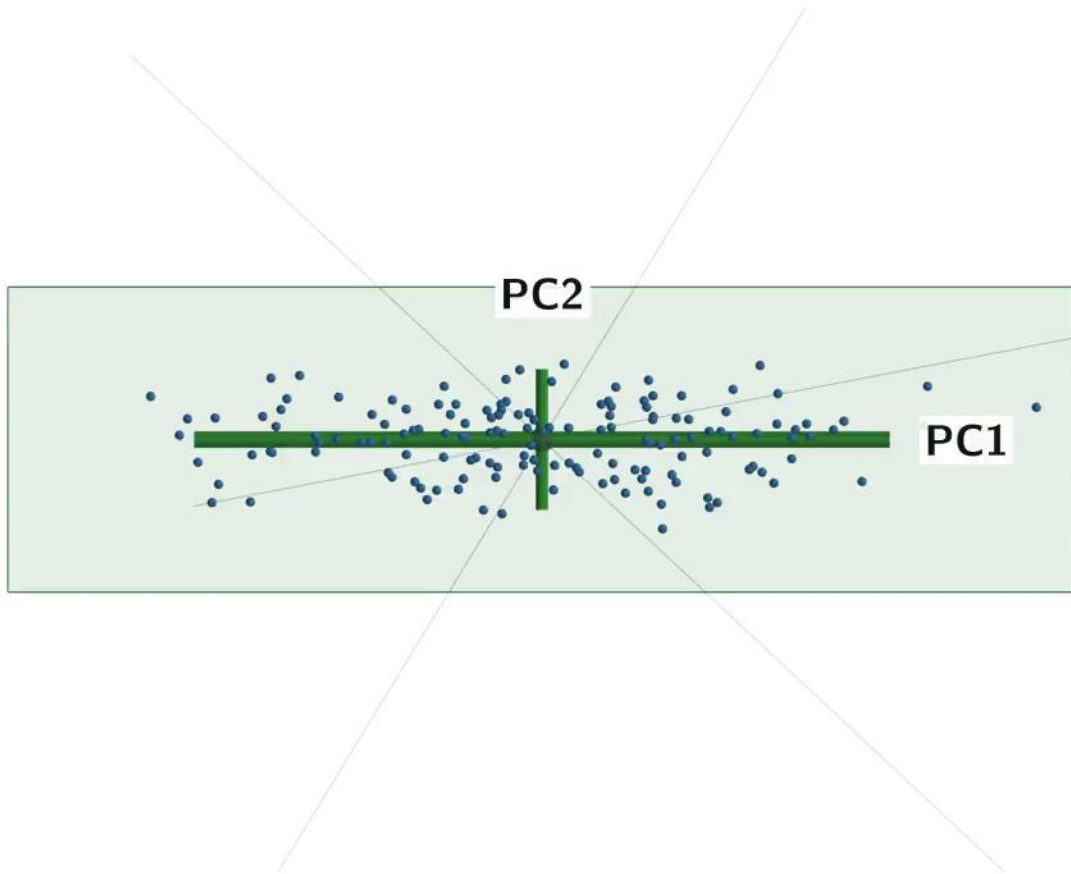


$$X_c V_{(2)} \in \mathbb{R}^{150 \times 2}, \quad V_{(2)} = [\mathbf{v}_1 \ \mathbf{v}_2]$$

Distancia al plano $\times 5$

Varianza perdida 0,3 %

Análisis de componentes principales



$$X_c V_{(2)} \in \mathbb{R}^{150 \times 2}, \quad V_{(2)} = [\mathbf{v}_1 \ \mathbf{v}_2]$$

Distancia al plano $\times 5$

Varianza perdida 0,3 %

$$\mathbf{v}_1 = \begin{pmatrix} 0,67 \\ 0,57 \\ 0,47 \end{pmatrix} \quad \mathbf{v}_2 = \begin{pmatrix} -0,63 \\ 0,11 \\ 0,77 \end{pmatrix}$$

PC1, carga de calefacción

PC2, gradiente a lo largo de la placa

Próxima clase

Hipótesis de hoy

- Entrenamiento, validación y prueba del mismo proceso
- Frontera de decisión lineal
- Estructura lineal de los datos, PCA

Clase 6, redes neuronales

- Capas lineales con funciones no lineales
- Entrenamiento, $\min_{\theta} L(\theta)$
- Redes informadas por la física (PINN)